Querying Cohesive Subgraphs in Temporal Graphs

Anonymous Author(s)

ABSTRACT

Temporal graphs are widely used to model dynamic interactions in various domains, including social networks, biology, and e-commerce. The identification of *cohesive subgraph models (CSMs)* within specified time windows in temporal graphs can uncover critical patterns and trends, which has received tremendous research attention. Existing studies focus on constructing index structures that support historical queries for particular types of CSMs, such as historical connected component (Historical-CC) and historical k-core (Historical-Core) queries. In this paper, we shift our focus towards a generalized class of *non-overlapping* and *monotone* CSMs, including connected component and k-core, and we study the corresponding query problem called the *Historical-CSM query*. We propose a *unified* algorithmic framework specifically designed to construct an index structure that facilitates queries for any given non-overlapping and monotone CSM. In particular, our method transforms a Historical-CSM query into a unified query problem, namely *spanning connected component query*, and introduces an index-based solution to efficiently answer this newly formulated query. We further demonstrate the flexibility of our approach by applying it to both Historical-CC and Historical-Core queries. Extensive experiments conducted on real-world graphs show that our method outperforms the state-of-the-arts by up to 60× faster for Historical-CC query and 8,000× faster for Historical-Core query.

ACM Reference Format:
Anonymous Author(s). 2018. Querying Cohesive Subgraphs in Temporal Graphs. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 19 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

Temporal graphs extend the classical definition of graphs by introducing the dimension of time [5, 33, 35, 48]. Unlike static graphs where edges are constant over time, temporal graphs capture the dynamics of interactions, allowing edges to appear and disappear at different timestamps. This temporal aspect allows modeling changes in connections over time, which is essential for understanding the evolving structure of networks in areas such as social interaction, financial transactions, biological system, and more.

The problem of efficiently answering queries for a graph snapshot within a specific time window is referred to as *historical queries*. Several types of historical queries have been explored in the literature [10, 24, 26, 32, 40–42, 49]. In this paper, we address the problem of *historical cohesive subgraph queries* on temporal graphs. The dynamic nature of temporal graphs presents challenges when querying cohesive subgraphs across different time intervals, as these subgraphs may evolve over time, with vertices/edges joining or leaving the subgraph at varying time intervals.

A variety of cohesive subgraph models (CSMs) have been studied in recent years [8, 16, 22, 31, 58], with some targeting temporal graphs [21, 28, 37, 54, 59]. In this paper, our focus lies on a broad category of CSMs, termed *non-overlapping* and *monotone* CSMs, which encompass a variety of cohesive subgraph models, such as connected components [54] and k-cores [59]. In particular, a CSM is said to be (1) non-overlapping if the vertex sets corresponding to different cohesive subgraphs in G are disjoint, and (2) monotone if all vertices in a cohesive subgraph remain in the same cohesive subgraph after adding vertices and/or edges to G. Given a non-overlapping and monotone CSM, our objective in this paper is to develop an index structure that enables us to efficiently retrieve all cohesive subgraphs for a given query time interval. This problem is referred to as the *historical non-overlapping and monotone CSM query* (or, Historical-CSM query) and is crucial for analyzing how cohesive groups of vertices change over time in temporal graphs. Tracking these subgraphs across different time intervals is essential for understanding the long-term behaviour of networks and identifying significant patterns or anomalies in temporal graphs.

Applications. Efficient querying of cohesive subgraphs in temporal graphs has broad applications across a variety of fields.

- Financial Anomaly Detection. By tracking cohesive subgraphs over time in financial networks, analysts can identify unusual patterns in the network, such as sudden changes in tightly integrated trading groups, which may signal fraudulent activities, market manipulation, and other forms of financial misconduct [4, 36].
- Temporal-Aware Recommendation. By incorporating the temporal evolution of user preferences, recommendation systems are able to offer more personalized suggestions [53]. Analyzing the changes in user interactions or preferences over time can lead to a better understanding of user behavior, thereby resulting in more relevant recommendations.
- Infectious Disease Tracking. Temporal graphs can be used to model the spread of infectious diseases by representing contact networks between individuals [13]. Cohesive subgraphs in these graphs might correspond to clusters of individuals who are more likely to spread disease within a certain period, providing key insights for controlling outbreaks.

Existing state-of-the-art solutions. A naive online solution to answer Historical-CSM queries involves first collecting all the edges whose timestamps fall within the query time interval, followed by running a CSM mining algorithm on the resulting graph. However, while simple, this online method becomes inefficient when dealing with large query time intervals or large graphs, as it requires accessing and processing all relevant edges which can be computationally expensive. Several existing studies have sought to

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, June 03–05, 2018, Woodstock, NY
© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/18/06…$15.00
https://doi.org/XXXXXXX.XXXXXXX

address these inefficiencies by proposing index-based solutions tailored to specific cohesive subgraph models; that is, these methods utilize the index structure to store useful information for supporting efficient historical queries with specific CSM [46, 54, 59], e.g., historical connected component (Historical-CC) and historical k-core (Historical-Core) queries. However, these indexing approaches are restricted to certain cohesive subgraph types, limiting their applicability to other cohesive subgraph models. Furthermore, the query processing of these index-based solutions typically involves iterating over a massive number of timestamps or vertices in the temporal graph. This requirement results in practical inefficiencies, especially when dealing with large-scale temporal graphs or those with low time densities in real applications.

Our unified framework. In this paper, we instead propose a more flexible and generalized solution by introducing a *unified* framework for efficiently querying historical non-overlapping and monotone CSMs. To tackle this problem, we introduce a novel concept called the *affiliated temporal spanning graph*, which can be used to capture the temporal evolution of cohesive subgraphs of the original temporal graph across various time intervals. As a result, answering a *spanning connected component query* on the affiliated temporal spanning graph is equivalent to solving the original Historical-CSM query. This transformation simplifies our problem and allows for a unified approach across different CSMs. To design an efficient index-based solution, we further introduce the concepts of *temporal connectivity forest* and *critical timestamp*. Utilizing these concepts, we can further transform the problem and focus on a specified three dimensional (3D) range search problem aimed at retrieving all 3D data points (x, y, z) that satisfy the condition $t_s \leq x \wedge y \leq t_e \leq z$ for a given query time interval $[t_s, t_e]$. Specifically, we propose a novel index structure, called the *binary tree with auxiliary substructures* ***(BTAS)***, that accommodates large-scale temporal graphs with extensive timestamps. We then apply the general structure of **BTAS** to Historical-CC and Historical-Core queries, achieving good theoretical and practical performances (see our experiments). For example, for the Historical-CC query, our method can answer a query in a logarithmic time complexity of $O(\log(|\mathcal{E}|) + k)$ using a linear space complexity of $O(|\mathcal{E}|)$, where $\mathcal{E}$ is the temporal edge set and k is the difference between the number of vertices and the number of found connected components.

Our contributions. Our main contributions are as follows:

- *Problem formulation.* We formulate a general query problem of querying historical non-overlapping and monotone cohesive subgraphs (Historical-CSM) in Section 2.
- *A unified index-based solution.* We propose a novel index structure **BTAS** designed to efficiently handle Historical-CSM queries for a broad class of CSMs in Section 3. Our index **BTAS** reduces the need to process a large amounts of data during queries, improving the performance query processing using a linear space complexity. Our unified framework is also applied to two key types of CSM queries, Historical-CC and Historical-Core queries.
- *Performance Evaluation.* We conduct extensive performance evaluations using real-world datasets in Section 4. The results show that our proposed method **BTAS** outperforms existing approaches in the majority of cases, achieving speedups up to 60× and 8,000× for Historical-CC and Historical core queries, respectively.

In addition, we discuss related work in Section 5 and conclude the paper in Section 6.

2 PRELIMINARIES

We focus on an undirected temporal graph $\mathcal{G} = (V, \mathcal{E})$, where V denotes the set of vertices and $\mathcal{E}$ denotes the set of *temporal* edges. Here, each temporal edge is denoted by a triple (u, v, t) with $u \in V$, $v \in V$ and $t \in \mathbb{N}$ indicating the timestamp when u interacts with v. W.l.o.g., we assume that all timestamps are consecutive integers varying from 1 to $t_{\max}$, thereby $t_{\max} \leq |\mathcal{E}|$. Given a time window $[t_s, t_e]$, the *projected graph* of $\mathcal{G}$ over $[t_s, t_e]$, denoted by $G_{[t_s,t_e]} = (V, E_{[t_s,t_e]})$, is an undirected graph that includes the set of vertices V and the set of edges $E_{[t_s,t_e]} = \{(u, v) \mid (u, v, t) \in \mathcal{E}, t \in [t_s, t_e]\}$. We remark that projected graphs ignore the temporal information. Given a graph $G = (V, E)$ and a vertex set $V_{sub} \subseteq V$, we use $G[V_{sub}]$ to denote the subgraph induced by V_{sub}, i.e., $G[V_{sub}]$ includes the set of vertices V_{sub} and the set of edges $\{(u, v) \in E \mid u, v \in V_{sub}\}$. We remark that all subgraphs considered in this paper are vertex-induced subgraphs.

Let $\mathcal{P}$ be a *cohesive (vertex-induced) subgraph model* (CSM) defined on an undirected graph, e.g., connected component, k-core, clique, k-VCC, and etc. Given an undirected graph $G = (V, E)$, we define $\mathcal{P}(G)$ by a set of (vertex-induced) subgraphs of G. For a subgraph g of G, if $g \in \mathcal{P}(G)$ (resp. $g \notin \mathcal{P}(G)$), we say that subgraph g satisfies (resp. violates) CSM $\mathcal{P}(G)$. In this paper, we focus on those CSMs that are *non-overlapping* and *monotone*, as below.

Definition 2.1 (Non-overlapping). A CSM $\mathcal{P}$ is said to be *non-overlapping* iff for a graph $G = (V, E)$ and any two subgraphs $G[V_1] \in \mathcal{P}(G), G[V_2] \in \mathcal{P}(G)$ with $V_1 \subseteq V, V_2 \subseteq V$, the vertex sets V_1 and V_2 are disjoint, i.e., $V_1 \cap V_2 = \emptyset$.

Clearly, for a non-overlapping CSM $\mathcal{P}$, $\mathcal{P}(G)$ consists of at most $|V|$ different subgraphs. For example, connected component belongs to a non-overlapping CSM and thus there are at most $|V|$ different connected components in a graph G.

Definition 2.2 (Monotone). A CSM $\mathcal{P}$ is said to be *monotone* iff given a graph $G = (V, E)$, for any subgraph $G[V_{sub}] \in \mathcal{P}(G)$ (with $V_{sub} \subseteq V$) and any super-graph $G' = (V', E')$ of G (with $V \subseteq V'$ and $E \subseteq E'$), there exists a subgraph $G'[V'_{sub}]$ in $\mathcal{P}(G')$ such that $V_{sub} \subseteq V'_{sub}$.

Intuitively, for a monotone CSM $\mathcal{P}$, all vertices in a subgraph satisfying $\mathcal{P}(G)$ remain in the same subgraph satisfying $\mathcal{P}(G)$ after adding additional vertices and/or edges to G. Besides, we note that quite a few CSMs satisfy the non-overlapping and monotone properties, including connected component, k-core, and k-VCC. To ease the writing, we refer CSMs to those satisfying the non-overlapping and monotone properties, by default.

Now, we are ready to formulate the problem studied in this paper.

Problem Definition (Historical Non-overlapping and Monotone Cohesive Subgraph Query). *Given a temporal graph $\mathcal{G}$, a time window $[t_s, t_e]$ and a non-overlapping and monotone CSM $\mathcal{P}$, the Historical-CSM query seeks to find all sets of vertices each of which S includes at least 2 vertices (i.e., $|S| \geq 2$) and induces a subgraph $G_{[t_s,t_e]}[S]$ in $\mathcal{P}(G_{[t_s,t_e]})$.*

We show an example of historical non-overlapping and monotone cohesive subgraphs across different time intervals in Figure 1. Specifically, each grid corresponds to a time interval and each cohesive subgraph is colored with the same color, e.g., there are two disjoint cohesive subgraphs in $[2, 5]$, i.e., $G[\{1, 2\}]$ and $G[\{3, 4, 5\}]$.

We remark that the Historical-CSM query has been widely studied recently. However, all focus on one specific type of CSM, i.e., Historical-k-Core (or, Historical-Core) and Historical-Connected-Component (or, Historical-CC). Thus, previous proposed methods cannot be generalized to query other CSMs. In the following, we will propose a *unified* framework to solve the Historical-CSM query and then apply the proposed method to answer Historical-Core and Historical-CC queries.

3 OUR UNIFIED INDEX-BASED FRAMEWORK

We then introduce our *unified* index-based framework, which constructs a novel index to support the Historical-CSM query. In general, it consists of two key components. The first one is a *transformer*, which transforms the Historical-CSM query w.r.t. various CSMs in the temporal graph to a unified query problem, namely *spanning connected component (Spanning-CC) query* in the *refined affiliated temporal graph* (Sections 3.1 and 3.2). Answering the later query solves the former problem equivalently. This guarantees the generality of our framework to answer different Historical-CSM queries. The second one corresponds to an index-based solution, namely the *Binary Tree with Auxiliary Substructures (BTAS)*, to answer the newly formulated Spanning-CC query (Sections 3.3 and 3.4).

3.1 Transforming Historical-CSM query to Spanning-CC query

Consider a temporal graph $\mathcal{G} = (V, \mathcal{E})$ and a CSM $\mathcal{P}$. Let u and v be two vertices in V. We say $[t_1, t_2]$ is a u-v time window (or simply, u-v-TW) w.r.t. $\mathcal{P}$ iff u and v belong to the same subgraph in $\mathcal{P}(\mathcal{G}_{[t_1,t_2]})$. Besides, it is said to be *minimal* iff for any subinterval $[t'_1, t'_2] \subset [t_1, t_2]$, u and v belong to the different subgraphs in $\mathcal{P}(\mathcal{G}_{[t'_1,t'_2]})$. Clearly, there could be multiple minimal u-v-TW between $[0, t_{max}]$. To illustrate, consider an example in Figure 1 where $[2, 3]$ is a minimal 3-4-TW and $[4, 4]$ is a minimal 3-5-TW.

Then, we observe the following lemma.

Lemma 3.1. *Let $[t_1, t_2]$ be a minimal u-v-TW w.r.t. $\mathcal{P}$. Any time window $[t'_1, t'_2]$ with $[t_1, t_2] \subseteq [t'_1, t'_2]$ is a u-v-TW w.r.t. $\mathcal{P}$.*

Proof. We can verify this lemma by the monotone property of $\mathcal{P}$ (Definition 2.2). □

Based on the minimal u-v-TWs, we define the *affiliated temporal spanning graph*, denoted by $\overline{\mathcal{G}}$, of $\mathcal{G}$ as below.

Definition 3.2 (Affiliated temporal spanning graph). Given a temporal graph $\mathcal{G} = (V, \mathcal{E})$ and a CSM $\mathcal{P}$, the *affiliated temporal spanning graph (or affiliated graph)* of $\mathcal{G}$ w.r.t. $\mathcal{P}$ is a temporal graph $\overline{\mathcal{G}} = (V, \overline{\mathcal{E}})$, which includes the set of vertices V and the set of spanning edges each of which corresponds to a minimal u-v-TW, formally,

$$\overline{\mathcal{E}} = \{(u, v, t_1, t_2) \mid u, v \in V, [t_1, t_2] \text{ is a minimal } u\text{-}v\text{-TW}\}. \quad (1)$$

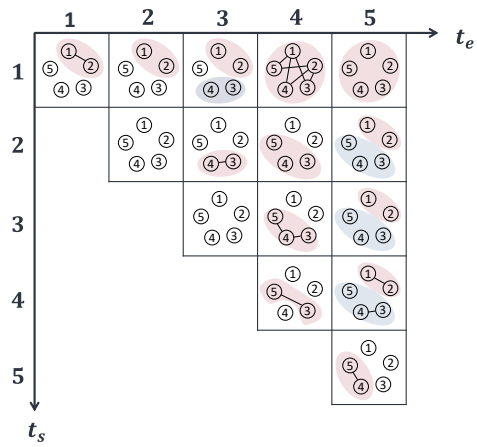


Figure 1: An example of historical non-overlapping and monotone CSM and the corresponding affiliated graph

To illustrate, see an affiliated graph in Figure 1. We note that each edge (u, v, t_1, t_2) in $\overline{\mathcal{G}}$ has two timestamps, namely t_1 and t_2, indicating the time interval $[t_1, t_2]$ where u interacts with v. We thus call it *spanning edge*. Clearly, a temporal edge (u, v, t) can be regarded as a special case of a spanning edge, i.e., (u, v, t, t). Given a time window $[t_s, t_e]$, the projected graph of $\overline{\mathcal{G}}$ over $[t_s, t_e]$, denoted by $\overline{G}_{[t_s,t_e]} = (V, \overline{E}_{[t_s,t_e]})$, is an undirected graph that includes the set of vertices V and the set of edges, i.e.,

$$\overline{E}_{[t_s,t_e]} = \{(u, v) \mid (u, v, t_1, t_2) \in \overline{\mathcal{E}}, [t_1, t_2] \subseteq [t_s, t_e]\}. \quad (2)$$

We observe that *two vertices u and v belong to the same subgraph in $\mathcal{P}(G_{[t_s,t_e]})$ if and only if u and v are adjacent in $\overline{G}_{[t_s,t_e]}$*. We verify this in two directions. First, if u and v belong to the same subgraph in $\mathcal{P}(G_{[t_s,t_e]})$, $[t_s, t_e]$ is a u-v-TW by definition and thus there exists at least one minimal u-v-TW $[t_1, t_2]$ such that $[t_1, t_2] \subseteq [t_s, t_e]$. Then, $\overline{\mathcal{G}}$ has a spanning edge (u, v, t_1, t_2), and thus u and v are adjacent in $\overline{G}_{[t_s,t_e]}$. Second, if u and v are adjacent in $\overline{G}_{[t_s,t_e]}$, $\overline{\mathcal{G}}$ has a spanning edge (u, v, t_1, t_2) such that $[t_1, t_2] \subseteq [t_s, t_e]$, which indicates a minimal u-v-TW $[t_1, t_2]$. By Lemma 3.1, we have that $[t_s, t_e]$ is a u-v-TW, and thus u and v belong to the same subgraph in $\mathcal{P}(G_{[t_s,t_e]})$ by definition. As a result, for all vertices belonging to the same subgraph in $\mathcal{P}(G_{[t_s,t_e]})$, they are adjacent to each other while not adjacent to other vertices (that are not in the subgraph) in $\overline{G}_{[t_s,t_e]}$, and thus they form a connected component of $\overline{G}_{[t_s,t_e]}$. Here, a connected component of a graph G is a connected (vertex-induced) subgraph that is not contained in any larger connected subgraph. Besides, for those vertices that do not appear in any subgraph in $\mathcal{P}(G_{[t_s,t_e]})$, each of them does not have any neighbours in $\overline{G}_{[t_s,t_e]}$ and thus corresponds to a connected component with the size equal to 1. We summarize the above fact below.

Lemma 3.3. *For a vertex subset S of V with $|S| \geq 2$, S induces a subgraph $G_{[t_s,t_e]}[S]$ in $\mathcal{P}(G_{[t_s,t_e]})$ iff S induces a connected component $\overline{G}_{[t_s,t_e]}[S]$ of $\overline{G}_{[t_s,t_e]}$.*

Based on the above lemma, *we can answer a Historical-CSM query by returning all vertex sets each of which S includes at least 2 vertices*

Algorithm 1: `Construction`$(\mathcal{P}, \mathcal{G})$

Input: The temporal graph $\mathcal{G} = (V, \mathcal{E})$ and a CSM $\mathcal{P}$
Output: An affiliated temporal spanning graph $\overline{\mathcal{G}}$

$\overline{\mathcal{E}} \leftarrow \emptyset$;
foreach *pair of vertices u and v in V* **do**
  $[t_s, t_e] \leftarrow [1, 1]$;
  while *t_e is no larger than $t_{\max}$* **do**
    $\mathcal{P}(G_{[t_s,t_e]}) \leftarrow$ `OnlineQ`$\langle\mathcal{P}\rangle(G_{[t_s,t_e]})$;
    if *u, v are in the same subgraph in $\mathcal{P}(G_{[t_s,t_e]})$* **then**
      while *$t_s \le t_e$ and u, v are in the same subgraph in $\mathcal{P}(G_{[t_s,t_e]})$* **do** $[t_s, t_e] \leftarrow [t_s + 1, t_e]$;
      $\overline{\mathcal{E}} \leftarrow \overline{\mathcal{E}} \cup \{(u, v, t_s - 1, t_e)\}$;
      $[t_s, t_e] \leftarrow [t_s + 1, t_e + 1]$;
    else
      $[t_s, t_e] \leftarrow [t_s, t_e + 1]$
return $\overline{\mathcal{G}}(V, \overline{\mathcal{E}})$;

and induces a connected component of $\overline{G}_{[t_s,t_e]}$. We define the later by the *Spanning-CC* query. We remark that the Spanning-CC query is *CSM-free*, i.e., it receives only two inputs $\overline{\mathcal{G}}$ and $[t_s, t_e]$. Hence, solving the Spanning-CC query helps to answer the Historical-CSM queries w.r.t. different CSMs.

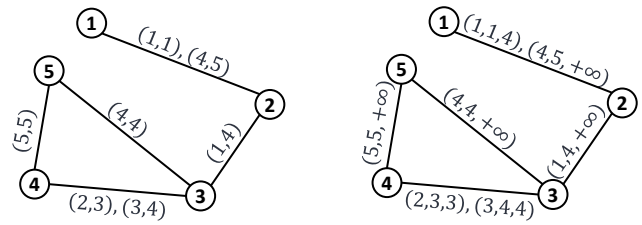
Historical-CC query v.s. Spanning-CC query. We note that the Historical-CC query can be regarded as a special case of the Spanning-CC query (since the former performs on a graph with temporal edges while the later performs on a graph with spanning edges). As a result, existing methods for answering Historical-CC queries cannot be adopted for solving the newly formulated Spanning-CC query. We will present an index-based method for the later in the following sections.

Construction of $\overline{\mathcal{G}}$. One remaining issue is *how to compute* $\overline{\mathcal{E}}$ for constructing $\overline{\mathcal{G}}$ based on $\mathcal{G}$ and $\mathcal{P}$. Let `OnlineQ`$\langle\mathcal{P}\rangle(G)$ be a procedure which receives a graph as the input and returns all CSMs in $\mathcal{P}(G)$. We note that its implementation has been widely studied for many CSMs, e.g., k-core [12, 23], (α,β)-core [29], and connected components [15]. Besides, we denote by $f(\mathcal{P}, G)$ the running time of `OnlineQ`$\langle\mathcal{P}\rangle(G)$. Based on this procedure, we present an algorithm `Construction` for constructing $\overline{\mathcal{G}}$ in Algorithm 1. The correctness can be easily verified. Besides, we can derive that (1) `Construction` runs in $O(|V|^2 \cdot t_{\max} \cdot f(\mathcal{P}, G))$ and (2) the number of spanning edges in $\overline{\mathcal{G}}$ is bounded by $|V|^2 t_{\max}$.

3.2 Refining an Affiliated Graph

We note that the affiliated temporal spanning graph $\overline{\mathcal{G}}$ constructed by Algorithm 1 could be very dense since the number of spanning edges is upper bounded by $O(|V|^2 \times t_{\max})$. As a result, the time and space costs of indexing on a dense graph would be very large. To reduce, we then introduce a method called `Reduction`, which refines the affiliated temporal spanning graph $\overline{\mathcal{G}}$ to a *sparser* one (by removing those redundant spanning edges) while keeping the results of all Spanning-CC queries unchanged.

We define the refined affiliated temporal spanning graph below.



(a) A refined affiliated graph **(b) With critical timestamps**

Figure 2: A refined affiliated graph of Figure 1

Definition 3.4. Let $\overline{\mathcal{G}} = (V, \overline{\mathcal{E}})$ be an affiliated graph. The *refined affiliated temporal graph (or refined affiliated graph)*, denoted by $\overline{\mathcal{G}'} = (V, \overline{\Psi})$, is a temporal graph where $\overline{\Psi}$ is the minimum edge set $\overline{\Psi} \subseteq \overline{\mathcal{E}}$, such that for any time window $[t_s, t_e]$, all Spanning-CCs of $\overline{G}_{[t_s,t_e]}$ have the same vertex sets as those of $\overline{G'}_{[t_s,t_e]}$, formally,

$$\forall \overline{G}_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(\overline{G}_{[t_s,t_e]}), \overline{G'}_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(\overline{G'}_{[t_s,t_e]}), \quad (3)$$

where $\mathcal{P}_{CC}$ is the connected component model.

Clearly, a Spanning-CC query on $(V, \overline{\Psi})$ would return the same vertex sets as that on $\overline{\mathcal{G}}$, while the former has *fewer* edges. Hence, we propose to refine $\overline{\mathcal{G}}$ to $(V, \overline{\Psi})$ as below.

Computation of $\overline{\Psi}$. Given a time window $[t_s, t_e]$, we define by $\overline{\mathcal{G}}_{\langle t_s,t_e\rangle} = (V, \overline{\mathcal{E}}_{\langle t_s,t_e\rangle})$ the *sliced graph* of $\overline{\mathcal{G}}$, formally, $\overline{\mathcal{G}}_{\langle t_s,t_e\rangle}$ includes the set of vertices V and the set of spanning edges, i.e.,

$$\overline{\mathcal{E}}_{\langle t_s,t_e\rangle} = \{(u, v, t_1, t_2) \mid (u, v, t_1, t_2) \in \overline{\mathcal{E}}, [t_1, t_2] \subseteq [t_s, t_e]\}. \quad (4)$$

We remark that, given time windows $[t_s, t_e]$ and $[t'_s, t'_e]$ such that $[t'_s, t'_e] \subseteq [t_s, t_e]$, the projected graph of sliced graph $\overline{\mathcal{G}}_{\langle t_s,t_e\rangle}$ over $[t'_s, t'_e]$ is $\overline{G}_{[t'_s,t'_e]}$, which is the same as that of $\overline{\mathcal{G}}$ over $[t'_s, t'_e]$.

We then observe the following lemma of the refined affiliated graph $(V, \overline{\Psi})$ of $\overline{\mathcal{G}}$.

Lemma 3.5. *Let $\overline{\mathcal{G}}$ be an affiliated graph and $(V, \overline{\Psi})$ be a refined one of $\overline{\mathcal{G}}$. For any time window $[t_s, t_e]$, $(V, \overline{\Psi}_{\langle t_s,t_e\rangle})$ is a refined affiliated graph of $\overline{\mathcal{G}}_{\langle t_s,t_e\rangle}$.*

Proof. This proof can be found in Appendix of [1]. □

We can easily verify that the graph shown in Figure 2(a) is a refined affiliated graph of Figures 1. Also, the spanning-CC query in Figure 2(a) thus corresponds to the Historical-CSM query in Figure 1.

In general, the computation of $\Psi_{\langle 0,t_{max}\rangle}$ follows a *dynamic programming* procedure. The idea is to (1) compute the set $\overline{\Psi}_{\langle t_s,t_e\rangle}$ based on two given sets $\overline{\Psi}_{\langle t_s+1,t_e\rangle}$ and $\overline{\Psi}_{\langle t_s,t_e-1\rangle}$ when $t_s < t_e$ and (2) directly compute $\overline{\Psi}_{\langle t_s,t_e\rangle}$ based on $\overline{\mathcal{G}}_{\langle t_s,t_e\rangle}$ when $t_s = t_e$. Below, we elaborate on the details.

Case 1: $t_s < t_e$. Given two sets $\overline{\Psi}_{\langle t_s+1,t_e\rangle}$ and $\overline{\Psi}_{\langle t_s,t_e-1\rangle}$, we note that

$$\overline{\Psi}_{\langle t_s,t_e\rangle} = \overline{\Psi}_{\langle t_s+1,t_e\rangle} \cup \overline{\Psi}_{\langle t_s,t_e-1\rangle} \cup \overline{\mathcal{E}}_{miss}, \quad (5)$$

where $\overline{\mathcal{E}}_{miss}$ is a set of edges that are in $\overline{\Psi}_{\langle t_s,t_e\rangle}$ but not in $\overline{\Psi}_{\langle t_s+1,t_e\rangle} \cup \overline{\Psi}_{\langle t_s,t_e-1\rangle}$. Clearly, each edge (u, v, t_1, t_2) in $\overline{\mathcal{E}}_{miss}$ has $t_1 = t_s$ and $t_2 = t_e$. Further, we note that $\overline{\Psi}_{\langle t_s,t_e\rangle}$ can be computed by iteratively

Algorithm 2: Reduction($\overline{\mathcal{G}}$)

Input: An affiliated graph $\overline{\mathcal{G}} = (V, \overline{\mathcal{E}})$
Output: A refined affiliated graph $(V, \overline{\Psi})$ of $\overline{\mathcal{G}}$

1 **foreach** $t_e \leftarrow 0, 1, \ldots, t_{\max}$ **do**
2 $\quad \overline{\Psi}_{\langle t_e, t_e \rangle} \leftarrow$ the edge set of an arbitrary spanning forest of $\overline{\mathcal{G}}_{[t_e, t_e]}$;
3 $\quad$ **foreach** $t_s \leftarrow t_e - 1, t_e - 2, \ldots, 0$ **do**
4 $\quad\quad \overline{\Psi}_{\langle t_s, t_e \rangle} \leftarrow \Psi_{\langle t_s+1, t_e \rangle} \cup \overline{\Psi}_{\langle t_s, t_e-1 \rangle}$;
5 $\quad\quad$ **foreach** $(u, v, t_s, t_e) \in \overline{\mathcal{E}}$ **do**
6 $\quad\quad\quad$ **if** *u and v are not connected in $\overline{\Psi}_{\langle t_s, t_e \rangle}$* **then**
7 $\quad\quad\quad\quad \overline{\Psi}_{\langle t_s, t_e \rangle} \leftarrow \overline{\Psi}_{\langle t_s, t_e \rangle} \cup \{(u, v, t_s, t_e)\}$
8 **return** $(V, \overline{\Psi}_{\langle 0, t_{max} \rangle})$;

adding to $\overline{\Psi}_{\langle t_s+1, t_e \rangle} \cup \overline{\Psi}_{\langle t_s, t_e-1 \rangle}$ each edge (u, v, t_s, t_e) in $\overline{\mathcal{E}}$ if u and v are not connected in the current $(V, \overline{\Psi}_{\langle t_s, t_e \rangle})$. Based on Lemma 3.5, it is no hard to verify the correctness, i.e., the obtained $(V, \overline{\Psi}_{\langle t_s, t_e \rangle})$ is a refined affiliated graph of $\overline{\mathcal{G}}_{\langle t_s, t_e \rangle}$. We remark that (1) the ordering of edges to be considered to add to $\overline{\Psi}_{\langle t_s+1, t_e \rangle} \cup \overline{\Psi}_{\langle t_s, t_e-1 \rangle}$ can be arbitrary, and (2) $\overline{\mathcal{E}}_{miss}$ may vary from different orderings of edges, which will not affect the computation of $\overline{\Psi}$.

Case 2: $t_s = t_e = t$. We note that $\overline{\Psi}_{\langle t, t \rangle}$ is a refined affiliated graph of $\overline{\mathcal{G}}_{\langle t, t \rangle}$ based on Lemma 3.5. Thus, it is easy to verify that $(V, \overline{\Psi}_{\langle t, t \rangle})$ is a spanning forest of $\overline{G}_{[t, t]}$. We remark that $\overline{G}_{[t, t]}$ could have various spanning forests and $(V, \overline{\Psi}_{\langle t, t \rangle})$ can be an arbitrary one.

Summary and analysis. We summarize the proposed method Reduction in Algorithm 2. Specifically, it computes $\overline{\Psi}_{\langle 0, t_{max} \rangle}$ by following a certain order, i.e., Line 1 and Line 3. The correctness can be verified based on the above discussion. Besides, we remark that the time complexity of Algorithm 2 is $O(\alpha(|\overline{\mathcal{E}}|) + \alpha(|V|)t_{\max}^2)$ where $\alpha(\cdot)$ is the inverse Ackerman function, which is dominated by the time costs of operations in Lines 4-7. Specifically, for each graph $(V, \overline{\Psi}_{\langle t_s, t_e \rangle})$, we maintain a disjoint-set data structure ([15]), denoted by $\mathcal{T}(\overline{\Psi}_{\langle t_s, t_e \rangle})$, for checking whether u and v are connected in $(V, \overline{\Psi}_{\langle t_s, t_e \rangle})$ (i.e., whether u and v come from the same set). We remark that each disjoint-set data structure with $|V|$ vertices is implemented by a forest of trees and thus checking whether two vertices are from the same set takes $O(\alpha(|V|))$ amortized time. It computes $\mathcal{T}(\overline{\Psi}_{\langle t_s, t_e \rangle})$ by merging $\mathcal{T}(\overline{\Psi}_{\langle t_s+1, t_e \rangle})$ and $\mathcal{T}(\overline{\Psi}_{\langle t_s, t_e-1 \rangle})$ (in Line 4), which takes $O(\alpha(|V|))$. Besides, for each edge (u, v, t_1, t_2) in $\overline{\mathcal{E}}$, it checks the connectivity of u and v in Line 6 exactly once, which takes $O(\alpha(\overline{\mathcal{E}}))$ in total.

3.3 Prerequisites of Our Index-based Solution

We note that Reduction in Algorithm 2 can compute a refined affiliated graph. This refined affiliated graph can be utilized to answer Spanning-CC queries, which correspond to Historical-CSM queries in the original temporal graph, in an online fashion. However, Algorithm 2 requires $O(\alpha(|\overline{\mathcal{E}}|) + \alpha(|V|)t_{\max}^2)$, where $\alpha(\cdot)$ is the inverse Ackerman function. This time complexity depends on the value of $t_{\max}$, which might be inefficient in practice. In order to design an index-based solution, we introduce the concepts of *temporal connectivity forest* and *critical timestamp* such that we can transform the Spanning-CC query problem to a conceptual query problem of 3-D range search. As a byproduct, we also compute a refined affiliated graph in $O(|\overline{\mathcal{E}}| \log |\overline{\mathcal{E}}|)$ during the transformation.

Temporal connectivity forest. Given a time interval, the Spanning-CC query for an affiliated graph $\overline{\mathcal{G}}$ can be easily obtained through its *temporal connectivity forest*, which is defined as follows.

Definition 3.6 (Temporal Connectivity Forest). Given an affiliated graph $\overline{\mathcal{G}}$ and a time interval $[t_s, t_e]$, a temporal subgraph $\varphi_{(t_s, t_e)}$ is said to be a *temporal connectivity forest* of $\overline{\mathcal{G}}$ if and only if the projected graph of $\varphi_{(t_s, t_e)}$ over $[t_s, t_e]$ is a spanning forest of $\overline{G}_{[t_s, t_e]}$, and $\varphi_{(t_s, t_e)}$ does not have any multiple edge.

Based on Definition 3.6, we know that the result of the Spanning-CC query in the affiliated graph $\overline{\mathcal{G}}$ for a query time interval $[t_s, t_e]$ corresponds to the temporal connectivity forest $\varphi_{(t_s, t_e)}$.

We next present a method to derive $\varphi_{(t_s, t_e)}$, which can be utilized to obtain the returned result of the Spanning-CC query problem with time interval $[t_s, t_e]$, from the refined affiliated graph $(V, \overline{\Psi})$. According to the definitions of refined affiliated graph (Definition 3.4) and temporal connectivity forest (Definition 3.6), given a refined affiliated graph $(V, \overline{\Psi})$, we know that there always exists a temporal connectivity forest $\varphi(t_s, t_e)$ of the affiliated graph $\overline{\mathcal{G}}$ such that $\varphi(t_s, t_e) \subseteq (V, \overline{\Psi}_{\langle t_s, t_e \rangle})$ for any time interval $[t_s, t_e]$. As a result, given a refined affiliated graph $(V, \overline{\Psi})$ and any query interval $[x, y]$ where $0 \le x \le y \le t_{\max}$, we can always obtain a temporal connectivity forest $\varphi(x, y)$ from $\overline{\Psi}$ by the following procedure.

(1) If $x = y$, we have that $\varphi_{(x,x)} \leftarrow (V, \overline{\Psi}_{\langle x, x \rangle})$;
(2) Otherwise, $\varphi_{(x,y)}$ is computed based on $\varphi_{(x+1,y)}$ and $\varphi_{(x,y-1)}$ as follows:
   (a) Set $\varphi_{(x,y)} \leftarrow \varphi_{(x+1,y)}$.
   (b) For each spanning edge $(u, v, t_1, t_2) \in \overline{\Psi}_{\langle x, y-1 \rangle}$, add it to $\varphi_{(x,y)}$ if u and v are not connected in $\varphi_{(x,y)}$.
   (c) For each spanning edge $(u, v, x, y) \in \overline{\Psi}$, add it to $\varphi_{(x,y)}$.

It is easy to see that we can correctly compute all the temporal connectivity forests by the above procedure. Moreover, these temporal connectivity forests have special properties as summarized in the following lemma.

Lemma 3.7. *The set of temporal connectivity forests $\{\varphi_{(t_s, t_e)} \mid 0 \le t_s \le t_e \le t_{\max}\}$ has the following properties.*

(1) $\forall (u, v, t_1, t_2) \in \overline{\Psi} \Rightarrow (u, v, t_1, t_2) \in \varphi(t_1, t_2)$.
(2) $\forall (u, v, t_1, t_2) \in \varphi(x+1, y) \Rightarrow (u, v, t_1, t_2) \in \varphi(x, y)$.
(3) $\forall (u, v, t_1, t_2) \notin \varphi(x, y-1) \Rightarrow (u, v, t_1, t_2) \notin \varphi(x, y)$.

Proof. It is easy to see that the first and second properties are due to Procedures (2c) and (2a), respectively. Moreover, the third property is the contrapositive of the second one. □

Critical timestamps. Suppose there is a spanning edge (u, v, t_1, t_2) such that it is in $\varphi(t_1, t_3)$ but not in $\varphi(t_1, t_3 + 1)$, we can conclude that (u, v, t_1, t_2) is in $\varphi_{(x,y)}$ for all $x \le t_1$ and $t_2 \le y \le t_3$ according to Lemma 3.7. Thus, we can derive the following definition.

Definition 3.8. Given a spanning edge $e = (u, v, t_1, t_2)$ in the refined affiliated graph $(V, \overline{\Psi})$, the *critical timestamp* t_3 of e is defined as the largest timestamp such that (u, v, t_1, t_2) exists in $\varphi(t_1, t_3)$.

Based on Definition 3.8, for each time interval $[t_s, t_e]$, we can recover the temporal connectivity forest $\varphi(t_s, t_e)$ based on spanning edges with their critical timestamps (u, v, t_1, t_2, t_3). That is, $\varphi(t_s, t_e)$ consists of spanning edges $\{(u, v, t_1, t_2) \mid t_s \leq t_1 \text{ and } t_2 \leq t_e \leq t_3\}$.

We now present the computation of the corresponding critical timestamp t_3 for each temporal edge (u, v, t_1, t_2). Note that this computation can be converted into the problem of maintaining the maximum spanning forest with online edge insertions in a weighted graph. Below, we elaborate on the details.

The transformed weighted graph has the same vertex/edge sets as the input affiliated graph $\overline{\mathcal{G}}$. We then assign weights to this graph: we iteratively assign non-decreasing weights (e.g., $1, 2, \ldots$) to spanning edges (u, v, t_1, t_2) in $\overline{\mathcal{G}}$, first by t_1 in non-decreasing order, breaking ties by t_2 in non-increasing order, while ensuring that spanning edges with identical (t_1, t_2) have the same weight.

Now, we first show that we can compute a refined affiliated graph of $(V, \overline{\Psi})$ as follows. In particular, our method iteratively processes the edges in $\overline{\mathcal{G}}$ according to the arrangement of: sort edges (u, v, t_1, t_2) in $\overline{\mathcal{G}}$ first by t_2 in non-decreasing order and break ties by t_1 in non-increasing order. In each iteration, we maintain a maximum spanning forest (where the corresponding edge set is dynamically maintained by the edge set S) with the graph constructed by all the considered edges in the ordering before the current iteration. Suppose we have just finished the iterations that consider all edges with t_1 of value x and t_2 of value y in $\overline{\mathcal{G}}$. Recall that we maintain a maximum spanning forest using S in each iteration, the addition of these edges with $(t_1, t_2) = (x, y)$ is solely depends on the presence of edges (p, q, t_s, t_e) with larger weights in S. According to the ordering in which we add edges, these edges must satisfy $x \geq t_s$ and $y \leq t_e$. This ensures that the edges we added are exactly $\overline{\mathcal{E}}_{miss}$ in Equation (5), aligning with the transition equation. Consequently, a refined affiliated graph $(V, \overline{\Psi})$ is thus computed.

Next, we show that we can also compute the corresponding critical timestamps t_3 for all edges in the computed affiliated graph of $(V, \overline{\Psi})$. Again, suppose we have just finished the iterations that consider all edges with t_1 of value x and t_2 of value y in $\overline{\mathcal{G}}$. We note that the newly added edges (with $(t_1, t_2) = (x, y)$), along with those edges of greater weights, form the edges of $\varphi_{(x,y)}$ by our construction. If an edge $(u, v, t_1, t_2) \in \varphi_{(x+1,y)}$, it cannot be replaced by edges with $t_s = x$ and $t_e = y$, since these edges have smaller weights, which aligns with the second property in Lemma 3.7. Moreover, if an edge is replaced by another whose $t_e = y$, it will not appear in the later iterations, thereby satisfying the third property in Lemma 3.7. Accordingly, the critical timestamp of the edge (u, v, t_1, t_2) is $y - 1$ according to Definition 3.8. In such way, we can correctly compute all the critical timestamps for spanning edges in $\overline{\Psi}$.

The above descriptions of the computation of a refined affiliated graph $(V, \overline{\Psi})$ of $\overline{\mathcal{G}}$ and critical timestamps for all spanning edges in $\overline{\Psi}$ are summarized in Algorithm 3. The correctness of Algorithm 3 follows the above discussion. We also note that the input graph for Algorithm 3 can be any subgraph S of the affiliated graph $\mathcal{G}'$ that satisfies Equation (3). For the time complexity of Algorithm 3,

Algorithm 3: `Index-Preparation`$(\overline{\mathcal{G}})$

Input: An affiliated graph $\overline{\mathcal{G}} = (V, \overline{\mathcal{E}})$
Output: A refined affiliated graph $(V, \overline{\Psi})$ of $(V, \overline{\mathcal{E}})$ and critical timestamps for all spanning edges in $\overline{\Psi}$

```
Assign non-decreasing weights to edges (u, v, t1, t2) in Ē,
  first by t1 in non-decreasing order, breaking ties by t2 in
  non-increasing order, while ensuring that edges with
  identical (t1, t2) have the same weight;
Sort edges (u, v, t1, t2) in Ē first by t2 in non-decreasing
  order and break ties by t1 in non-increasing order;
Initialize S ← ∅ and Ψ̄ ← ∅;
foreach e = (u, v, t1, t2) ∈ Ē do
    if u, v are connected in S then
        e* ← the edge with minimum weight in the cycle
          connecting u and v;
        if weight of e is greater than that of e* then
            S ← S ∪ {e} \ {e*};
            Ψ̄ ← Ψ̄ ∪ {e*};
            Set the critical timestamp t3 ← t2 − 1 for e*;
    else
        S ← S ∪ {e};
Set critical timestamps t3 ← +∞ for remaining edges in S;
Ψ̄ ← Ψ̄ ∪ S;
```

we remark that the algorithm follows the standard fashion of the problem of incremental maximum spanning forest maintenance, which can be solved with a link-cut tree structure using the implementation of common binary trees and splay trees [43]. Thus, the overall time complexity of Algorithm 3 is $O(|\overline{\mathcal{E}}| \log |\overline{\mathcal{E}}|)$.

A conceptual query problem. Based on the concepts of temporal connectivity forest (Definition 3.6) and critical timestamp (Definition 3.8), we can consider another conceptual query problem that is equivalent to the Spanning-CC query in the refined affiliated temporal graph. Specifically, for each edge in (u, v, t_1, t_2) in the refined affiliated temporal graph $\overline{\Psi}$, we can store a data point (t_1, t_2, t_3) in a three dimensional (3-D) space, where t_1, t_2, and t_3 correspond to the x, y, and z coordinates, respectively. Given an input query time interval $[t_s, t_e]$ of a Spanning-CC query, computing a spanning forest $\varphi_{(t_s,t_e)}$ is equivalent to querying the set of data points in the 3-D space that meet the condition $t_s \leq t_1 \wedge t_2 \leq t_e \leq t_3$.

Thus, based on the above discussion, we formally have the following equivalent problem.

Problem Definition (3-D Range Query Problem with Structural Properties). *We are given a set of 3-D data points $P = \{(x_1, y_1, z_1), (x_2, y_2, z_2), \ldots, (x_n, y_n, z_n)\}$, where each data point (x, y, z) in P satisfying $x \leq y \leq z$. We aim to design an index structure that enables queries to return the set of data points that meet the condition $q_1 \leq x \leq y \leq q_2 \leq z$ for any given query time interval $[q_1, q_2]$.*

Note that the number n of data points in P is upper bounded by the number of spanning edges in the refined affiliated temporal graph $\overline{\Psi}$, i.e., $n \leq |\overline{\Psi}| \leq |\overline{\mathcal{E}}|$.

3.4 Our Index Structure: BTAS

In this section, we introduce our index structure designed to answer the 3-D range query problem with structural properties (defined in Section 3.3), which, in turn, solves the Historical-CSM query.

Index structure. Our index structure T, called *Binary Tree with Auxiliary Substructures (**BTAS**)*, is a binary tree. In the **BTAS** T, we recursively partition the whole data point set P into smaller subsets, with each subset of points maintained in the corresponding tree node. Specifically, the **BTAS** T can be defined recursively as follows (See Figure 3 for illustration).

(1) If the data point set $P = \emptyset$, **BTAS** T is an empty tree;

(2) Otherwise, we now have $P \neq \emptyset$. Before the description, we first provide the following definitions. Let med be the median of the set of numbers $\{y_1, \ldots, y_n, z_1, \ldots, z_n\}$ (i.e., the median of all y-coordinates and z-coordinates of data points in P). Based on the value of med, we can partition the data points in P into three disjoint sets of data point P_{left}, P_{mid}, and P_{right}:

$$P_{left} := \{(x, y, z) \in P \mid z < med\},$$
$$P_{mid} := \{(x, y, z) \in P \mid y \leq med \leq z\},$$
$$P_{right} := \{(x, y, z) \in P \mid y > med\}.$$

Let the root node of the current **BTAS** T be v. Then, we can define:

- the left sub-tree T_{left} of v is a **BTAS** tree associated with the set of data points P_{left};
- the right sub-tree T_{right} of v is a **BTAS** tree associated with the set of data points P_{right}.

Moreover, the root node v stores the median med and is associated with the set of P_{mid} using two auxiliary substructures:

- The first substructure T_{xy} facilitates querying 2-D data points in the set $\{(x, y) \mid (x, y, z) \in P_{mid}\}$, within the range $q_1 \leq x \leq q_2$ and $y \leq q_2$.
- The second substructure T_{xz} facilitates querying 2-D data points in the set $\{(x, z) | (x, y, z) \in P_{mid}\}$, within the range $q_1 \leq x \leq q_2$ and $q_2 \leq z$.

We note that the two auxiliary substructures in **BTAS** can be implemented using the well-known *priority search tree*. In particular, a priority search tree takes input as three parameters q_x, q'_x, q_y and returns data points (x, y) that satisfy $q_x \leq x \leq q'_x$ and $q_y \leq y$ in $O(\log n + k)$ time, where n is the total number of data points stored in the priority search tree and k is the number of reported data points [34].

We remark that it is easy to see that the **BTAS** T is a balanced search tree, since **BTAS** makes use of med of points P to generate two subsets, P_{left} and P_{right}, of relatively equal size for the sub-trees. Thus, it is easy to see that the height of a **BTAS** is $O(\log n)$.

Query processing. Based on the **BTAS** structure, we can efficiently answer the queries of our conceptual problem in this section. In particular, given a query time interval $[q_1, q_2]$ and a **BTAS** tree T associated the set P of data points, we aim to find data points (x, y, z) that satisfy the condition $q_1 \leq x \leq y \leq q_2 \leq z$.

Following the **BTAS** structure T, the query processing descends along a specific path of tree nodes in T. Along this path, we aggregate the sets of data points that meet the required condition in both tree nodes and auxiliary substructures, and the resulting set is returned upon reaching the leaf of T. Specifically, the query

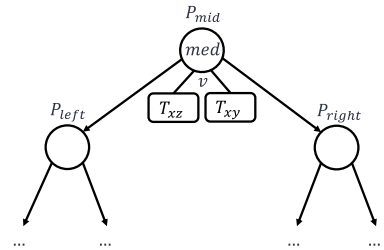


Figure 3: The recursive structure of BTAS

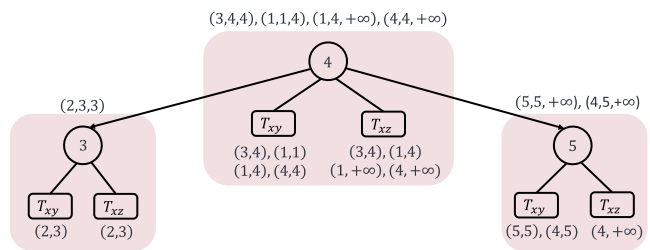


Figure 4: A running example of BTAS based on Figure 2(b)

on **BTAS** is processed recursively. Recall that the value of med is stored in the root of the current **BTAS** T, which is used to partition the set of data points P into three parts, i.e., P_{left}, P_{right}, and P_{mid}. Thus, we try to find out the resulting set in these three sets of data points. Given a query interval $[q_1, q_2]$, we have the following cases.

(1) We first consider P_{left} and P_{right} in sub-trees of the current T.
   (a) If $q_2 < med$, we search along the left sub-tree since any data point in P_{right} satisfies $y > med > q_2$ and thus violates the query condition $y \leq q_2$.
   (b) If $q_2 > med$, we search along the right sub-tree since any data point in P_{left} satisfies $q_2 > med > z$ and thus violates the query condition $q_2 \leq z$.
   (c) If $q_2 = med$, the data points in the final result set cannot lie in either sub-tree, and we handle these data points in the auxiliary substructures.

(2) We next consider P_{mid} and its two auxiliary substructures in the root node of the current T.
   (a) If $q_2 \leq med$, we know that $q_2 \leq z$ is already satisfied for all data points in P_{mid}. Thus, we focus on the query condition $q_1 \leq x \leq y \leq q_2$, and obtained the result of related data points using the auxiliary substructure T_{xy}.
   (b) If $q_2 > med$, we know that $y \leq q_2$ is already satisfied for all data points in P_{mid}. Thus, we focus on the query condition $q_1 \leq x \wedge q_2 \leq z$, and obtained the result of related data points using the auxiliary substructure T_{xz}.

To illustrate the above procedure, suppose that we have the refined affiliated graph (Figure 2) and $P = \{(2, 3, 3), (3, 4, 4), (1, 1, 4), (1, 4, +\infty), (4, 4, +\infty), (4, 5, +\infty), (5, 5, +\infty)\}$. The corresponding **BTAS** is in Figure 4 and we are searching CSMs in time interval $[2, 5]$. By the query processing, we first compare med stored in the current root node (which is 4) with q_2 (which is 5). Since $q_2 > med$, we search for T_{xz} with the query condition $2 \leq x \wedge 5 \leq z$, the

Algorithm 4: `BTAS-Query`

Input: The **BTAS** structure T and a query interval $[q_1, q_2]$
Output: The result set R of data points

1 $R \leftarrow \emptyset$;
2 **if** $T = \emptyset$ **then return** R;
3 $med \leftarrow$ the median value stored in root node of T;
4 **if** $q_2 \leq med$ **then**
5 $\quad R \leftarrow R \cup$ `PST-Query`(T_{xy}, q_1, q_2);
6 $\quad R \leftarrow R \cup$ `BTAS-Query`(T_{left}, q_1, q_2);
7 **else**
8 $\quad R \leftarrow R \cup$ `PST-Query`(T_{xz}, q_1, q_2);
9 $\quad R \leftarrow R \cup$ `BTAS-Query`(T_{right}, q_1, q_2);
10 **return** R;

result is $(4, 4, +\infty)$. Then, we search along the right sub-tree since $q_2 > med$. Subsequently, we search for T_{xy} following the query processing, and obtain the result $(5, 5, +\infty)$ and $(4, 5, +\infty)$. Thus, the final result set contains $(4, 4, +\infty)$, $(5, 5, +\infty)$, and $(4, 5, +\infty)$, which corresponds to the edge Set $\{(3, 5), (4, 5), (1, 2)\}$. Thus, we conclude Historical-CSMs in $[2, 5]$ are $G[\{1, 2\}]$ and $G[\{3, 4, 5\}]$.

We summarize the above query processing procedure of **BTAS** in Algorithm 4. Note that we make use of the well-known priority search tree [34] to implement the auxiliary substructures; thus, in Lines 5 and 8 of Algorithm 4, the query processing algorithm of priority search tree `PST-Query` is utilized to conduct queries in the two auxiliary substructures T_{xy} and T_{xz} (for completeness, we present the query processing algorithm in Appendix of the technical report [1]). The correctness of the query processing of **BTAS** is easy to see, as the procedure follows a specific path of tree nodes in T, with the remaining tree nodes in T do not contain any valid data points in the solution. We next analyze the query time complexity.

Lemma 3.9. *Our query processing of **BTAS** (Algorithm 4) requires $O(k + \log^2 n)$ time, where k is the number of returned data points.*

Proof. As mentioned, the query processing descends along a specific path of tree nodes in a **BTAS** T. Moreover, the height of a **BTAS** T is $O(\log n)$. Thus, Algorithm 4 visits $O(\log n)$ tree nodes in T. For each tree node, the most time-consuming part is to perform the query processing of the priority search tree `PST-Query`, which takes $O(\log n)$ time [34]. Thus, the total complexity is $O(\log^2 n + k)$, where k is the number of returned data points. □

Index construction. Given a set P of 3-D data points, the construction of a **BTAS** tree directly follows the defined recursive structure of **BTAS**. The construction algorithm is then summarized in Algorithm 5. Specifically, in Line 3, we compute the median *med* of all y-coordinates and z-coordinates of data points in P. Then, we use the value of *med* to partition the data points in P into three disjoint sets of data points, namely P_{left}, P_{mid}, and P_{right}, in Lines 4-6. Then, Lines 7-8 recursively construct the left and right **BTAS** sub-trees of the current **BTAS** tree T, which are associated with the data point sets P_{left} and P_{right}, respectively. For P_{mid}, **BTAS** makes use of two auxiliary substructures T_{xy} and T_{xz}, which support querying 2-D data points in the sets $\{(x, y) \mid (x, y, z) \in P_{mid}\}$ and $\{(x, z) \mid (x, y, z) \in P_{mid}\}$, respectively. In our implementation,

Algorithm 5: `BTAS-Construction`

Input: The set P of n 3-D data points
Output: A **BTAS** tree T

1 $T \leftarrow \emptyset$;
2 **if** $P = \emptyset$ **then return** T;
3 $med \leftarrow$ the median of all y-coordinates and z-coordinates of data points in P, i.e., $\{y_1, \ldots, y_n, z_1, \ldots, z_n\}$;
4 $P_{left} \leftarrow \{(x, y, z) \in P \mid z < med\}$;
5 $P_{mid} \leftarrow \{(x, y, z) \in P \mid y \leq med \leq z\}$;
6 $P_{right} \leftarrow \{(x, y, z) \in P \mid y > med\}$;
7 left **BTAS** sub-tree of $T \leftarrow$ `BTAS-Construction`(P_{left});
8 right **BTAS** sub-tree of $T \leftarrow$ `BTAS-Construction`(P_{right});
9 $T_{xy} \leftarrow$ `PST-Construction`$(\{(x, y) \mid (x, y, z) \in P_{mid}\})$;
10 $T_{xz} \leftarrow$ `PST-Construction`$(\{(x, z) \mid (x, y, z) \in P_{mid}\})$;
11 **return** T;

we make use the well-known priority search tree [34]. Thus, in Lines 9-10, the two auxiliary substructures T_{xy} and T_{xz} are constructed using the construction algorithm of priority search tree (for completeness, we present the construction algorithm in Appendix of the technical report [1]).

We next show the index construction time of `BTAS-Construction` (Algorithm 5) and the space complexity of the **BTAS** structure.

Lemma 3.10. *The index construction time complexity of the **BTAS** structure requires $O(n \log n)$. Moreover, the space complexity of the **BTAS** structure is $O(n)$.*

Proof. This proof can be found in Appendix of [1]. □

Remark. We remark that our 3-D range query problem with structural properties (defined in Section 3.3) can also be solved using the data structures for the 3-D orthogonal query problem. For example, a 3-D range tree can support queries of the form $q_{x1} \leq x \leq q_{x2} \wedge q_{y1} \leq y \leq q_{y2} \wedge q_{z1} \leq z \leq q_{z2}$ in $O(k + \log^2 n)$ time and $O(n \log(\frac{\log n}{\log \log n})^2)$ space [9]. On the other hand, our querying problem has the structural property (specially, we have the constraint $t_s \leq x \leq y \leq t_e \leq z$), which makes our novel index solution **BTAS** can support queries in $O(k + \log^2 n)$ time and $O(n)$ space. We believe that this might be of independent theoretical interest.

3.5 Summary and Example Historical-CSM Queries

Given a temporal graph $\mathcal{G}$ and a non-overlapping and monotone CSM $\mathcal{P}$, we can develop an index-based solution for the Historical-CSM query w.r.t. $\mathcal{P}$, as described in Sections 3.1, 3.2, 3.3, and 3.4. In particular, our unified index-based solution **BTAS** is built as follows. We first utilize Algorithm 3 to compute a refined affiliated graph with critical timestamps (Section 3.3). Subsequently, we use the index construction algorithm of **BTAS** (Algorithm 5) to build our index structure in Section 3.4. We stress that the input graphs for Algorithm 3 vary depending on the specific CSMs, $\mathcal{P}$. Specifically, for a CSM $\mathcal{P}$, the input graph $\mathcal{G}'$ of Algorithm 3 should satisfy

$$\forall \overline{G}_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(\overline{G}_{[t_s,t_e]}), G'_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(G'_{[t_s,t_e]}),$$

where $\overline{G}$ is the affiliated graph. Below, we provide the details of our index-based solution specified for two example CSMs: connected components and k-core.

- Historical-CC query. As the input temporal graph $\mathcal{G}$ satisfies the above condition, we can directly use $\mathcal{G}$ as the input of Algorithm 3. Then, we can directly build the corresponding index following the procedure in Section 3.4. The resulting index is called **BTAS-CC**.
- Historical-Core. The input graph of Algorithm 3 can be computed by existing historical core enumeration algorithms, e.g.,[55, 59]. In our implementation, we use the algorithm in [59] due to its practical performance. Again, we can follow the same procedure in Section 3.4, and the resulting index is called **BTAS-Core**.

Index maintenance. Our index-based structure supports dynamic updates. It is easy to see that we need to deal with the update of the refined affiliated graph with critical timestamps and the corresponding **BTAS**. Specifically, when edges with a new timestamp of $t_{\max} + 1$ arrive, all newly added edges in the affiliate graph $\overline{\mathcal{G}}$ share the same end timestamp $t_e = t_{\max} + 1$. Given that Algorithm 3 processes edges in ascending order based on their end timestamps t_e, restoring the state of the algorithm allows for extension of its execution to $t_e = t_{\max} + 1$. Fortunately, Algorithm 3 maintains a maximum spanning forest over the interval $[1, t_{\max}]$, which represents the query result for the same interval. Consequently, the refined affiliated graph and critical timestamps can be dynamically updated. In Line 10 of Algorithm 3, the critical timestamp for a spanning edge may change from $+\infty$ to $t_{\max}$. These changes must be accordingly updated in the **BTAS** index structure. Further, if a new edge with $t_e = t_{\max} + 1$ integrates into the refined affiliated graph, it is added to **BTAS** along with its designated critical timestamp.

4 EXPERIMENTS

In this section, we conduct extensive experiments to evaluate the performance of our proposed method **BTAS**. In particular, we adapt our unified framework to Historical-CC and Historical-Core queries (described in Section 3.5) and compare **BTAS-CC** and **BTAS-Core** with the state-of-the-art approaches **TSF** [54] (for Historical-CC) and **PHC** [59] (for Historical-Core), respectively. All algorithms are implemented in C++, compiled with g++ -O2, and run on a Linux machine with an Intel Xeon 2.60GHz CPU and 256GB RAM. Our source codes are available in [1].

Datasets. We evaluate the algorithms on datasets[1] that are widely used in previous studies. We note that the datasets used in [59] and [54] are covered. The statistics of all datasets are shown in Table 1.

4.1 Experiments for Historical-CC Queries

We evaluate the performance of Historical-CC algorithms in terms of query processing, index construction, and index maintenance.

Query processing. To evaluate the performance of query processing, we generate 10,000 random queries with different time intervals of $[t_s, t_e]$, i.e., both t_s and t_e are selected randomly. The average time cost per query on each dataset is shown in Figure 5. We have the following observations. **First**, our approach **BTAS-CC** is generally faster and achieves up to 20× speedups (on the EE and MI datasets) compared with the state-of-the-art **TSF** index. Moreover, we note that the results of the **TSF** index on the WT and SO datasets are marked as **N/A** since we are unable to complete the index construction within 24 hours.

[1]These datasets can be found in the SNAP (https://snap.stanford.edu/data/) and KONECT projects (http://konect.cc/networks/).

Datasets	$\lvert V\rvert$	$\lvert\mathcal{E}\rvert$	$t_{\max}$
contact (CT)	275	28,244	15,662
ColledgeMsg (CM)	1,862	59,835	58,910
sx-superuser (SU)	159,481	144,339	1,437,198
Emain-E (EE)	966	332,334	207,879
sx-mathoverflow (MO)	21,980	506,550	505,783
facebook-wosn-links (FL)	63,732	817,035	333,924
sx-askubuntu (AU)	127,047	964,437	960,865
mit (MI)	97	1,086,404	33,452
wiki-talk-temporal (WT)	1,120,716	7,833,140	7,375,041
youtube-u-growth (YG)	3,223,590	9,375,374	203
wikipedia-growth (WG)	1,870,710	39,953,145	2,198
sx-stackoverflow (SO)	2,296,666	63,497,050	52,423,236

Table 1: Statistics of all datasets

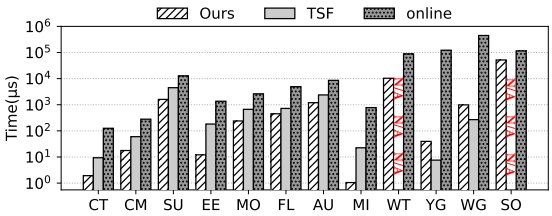
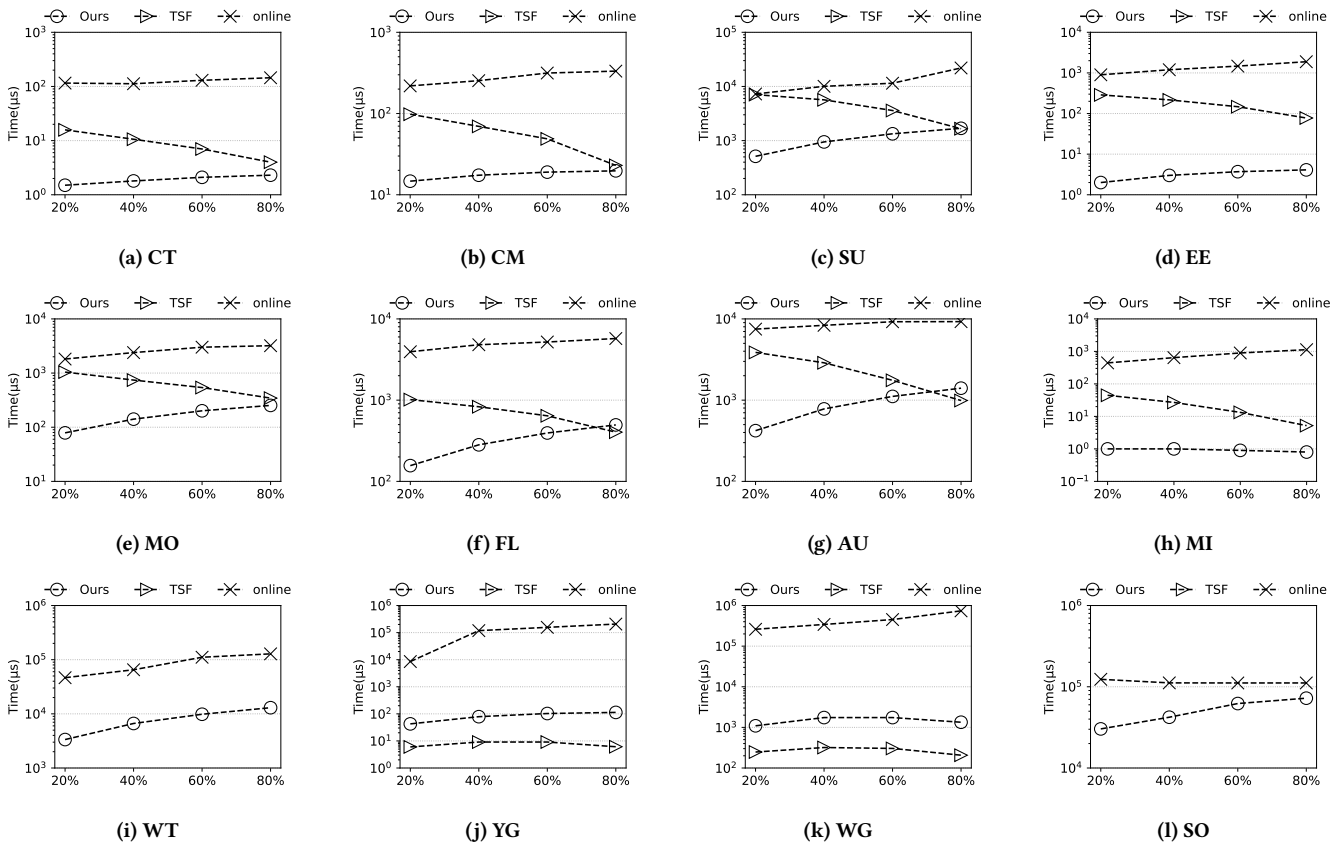


Figure 5: Average time cost per query for Historical-CC

Besides, the **online** method does not need to construct the related index but shows the worst performance in terms of query processing. **Second**, the **TSF** index has a better performance on the YG and WG datasets. This may be due to the fact that the largest values of timestamp $t_{\max}$ in these two datasets — 203 for YG and 2,198 for WG — are relatively small compared with the scale of datasets (YG and WG have 9,375,374 and 39,953,145 temporal edges, respectively). In such cases, iterating through timestamps in the **TSF** approach becomes less time-consuming.

We also evaluate the time cost per query against different query time intervals, i.e., $t_e - t_s$, in Figure 6. Specifically, we set the interval sizes to as 20%, 40%, 60%, and 80% of $t_{\max}$ and record the average running time of 10,000 queries with each query interval size. We can see that our method **BTAS-CC** outperforms **TSF** and **online** in most cases, particularly when the size of query interval is small. For example, our method achieves up to 60× speedups on the EE and MI datasets when the interval size is 20% of $t_{\max}$. However, as the time interval size increases, the performance gap narrows, where **TSF** has a better performance on the query interval of 80%$t_{\max}$ in some datasets. This is because the query performance of **TSF** depends on the value of $t_{\max}$ and performs better when the query interval is large, say at least 80%$t_{\max}$. On the other hand, our approach **BTAS-CC** is suitable for different query time intervals.



(a) CT (b) CM (c) SU (d) EE

(e) MO (f) FL (g) AU (h) MI

(i) WT (j) YG (k) WG (l) SO

Figure 6: Historical-CC query time by varying query time interval sizes

Index construction and maintenance. We first compare the efficiency of index construction on all datasets in Figure 7. As mentioned, the **TSF** method cannot construct the index within 24 hours on the WT and SO datasets. From Figure 7, it is clear to see that in most cases, our index construction method outperforms the **TSF** in terms of running time. In particular, we can construct our **BTAS-CC** index in 7.5 seconds on the WT dataset, while we are unable to build the **TSF** index even after spending more than 24 hours. The performance for index construction is in line with the theoretical result that the construction time of our method costs $O(|\mathcal{E}| \log |\mathcal{E}|)$ while that of **TSF** requires $O(|\mathcal{E}| \cdot t_{\max})$. Besides, we report the space consumption of different methods in Figure 8. We can see that these two index structures **BTAS-CC** and **TSF** occupy nearly the same amount of space across most datasets, which is also implied by the same space complexity of $O(|\mathcal{E}|)$ for both index structures.

We also evaluate the time cost of index maintenance when new edges are arrived. Specifically, following the previous work [54], we build the index first with edges in $[0, 0.8t_{\max})$, then update the index with the remaining edges. The time cost is illustrated in Figure 9. In general, the dynamic update of our proposed index **BTAS-CC** is generally more efficient than that of the **TSF** index (and the **TSF** index cannot be constructed in 24 hours on the WT and SO datasets). We can see in Figure 9 that the average update time of **BTAS-CC**

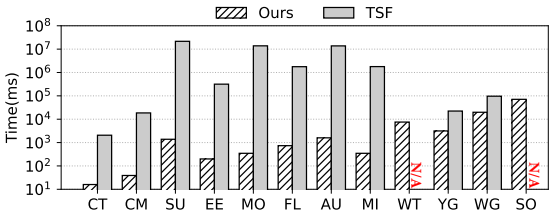


Figure 7: Index construction time for Historical-CC

is up to two orders of faster than the **TSF** index. The reason is as follows. When a new time stamp $t_{\max} + 1$ is appended, our method takes $O(\tilde{m} \cdot \log(|V| + \tilde{m}))$ time to update the index structure, where $\tilde{m}$ is the number of newly arrived temporal edges associated with $t_{\max} + 1$, while **TSF** takes $O(\tilde{m} \cdot t_{\max})$ update time.

4.2 Experiments for Historical-Core Queries

We evaluate the performance of Historical-Core algorithms in terms of query processing and index construction.

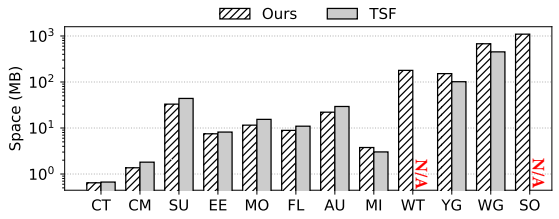


Figure 8: Index space consumption for Historical-CC

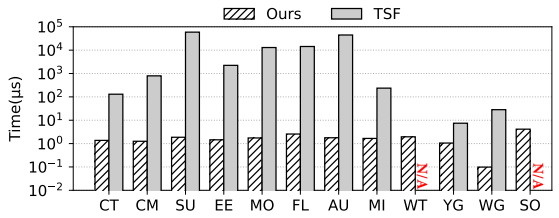


Figure 9: Average update time per edge for Historical-CC

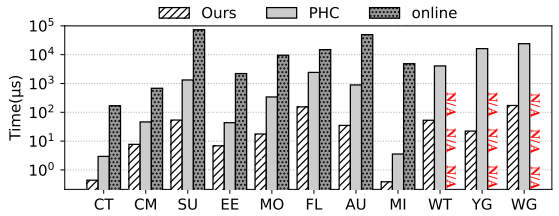


Figure 10: Average time cost per query with $k = 30\% k_{\max}$ for Historical-Core

Query processing. We investigate the performance of query processing by varying the size of the input query time interval. Specifically, we first evaluate the average time cost for queries with 100,000 randomly selected pairs of t_s and t_e when $k = 30\% k_{\max}$ in Figure 10. From the results, we can see that (1) the **online** method cannot complete 100,000 queries in 12 hours for the WT, YG, and WG datasets; and (2) for the YG dataset, our method **BTAS-Core** runs 730× faster than the state-of-the-art **PHC** index.

Then, we vary the query interval size as 20%, 40%, 60%, and 80% of $t_{\max}$, and set the parameter k as 10%, 30%, 50%, 70% and 90% of $k_{\max}$. For each combination of query interval size and k, we randomly generate 100,000 queries. Figure 11 shows the average time cost per query for $k = 30\% k_{\max}$ (the results for other values of k are provided in Appendix of [1]). From Figure 11, we can observe that our method **BTAS-Core** outperforms the state-of-the-art Historical-Core method **PHC** and the **online** method in all instances. For example, **BTAS-Core** outperforms **PHC** by at least 100 times across all parameter combinations tested on the YG dataset. Moreover, the query time of **BTAS-Core** increases as the interval size increases. This is because a larger interval size involves more vertices in the k-core, requiring **BTAS-Core** to search through more tree nodes in a **BTAS-Core** tree. We next conduct the experiments by varying the values of k on the FL and SU datasets in Appendix of [1]. We can conclude that the query time of all indices decreases monotonically with increasing k, since a larger k implies fewer vertices in the resulting k-core.

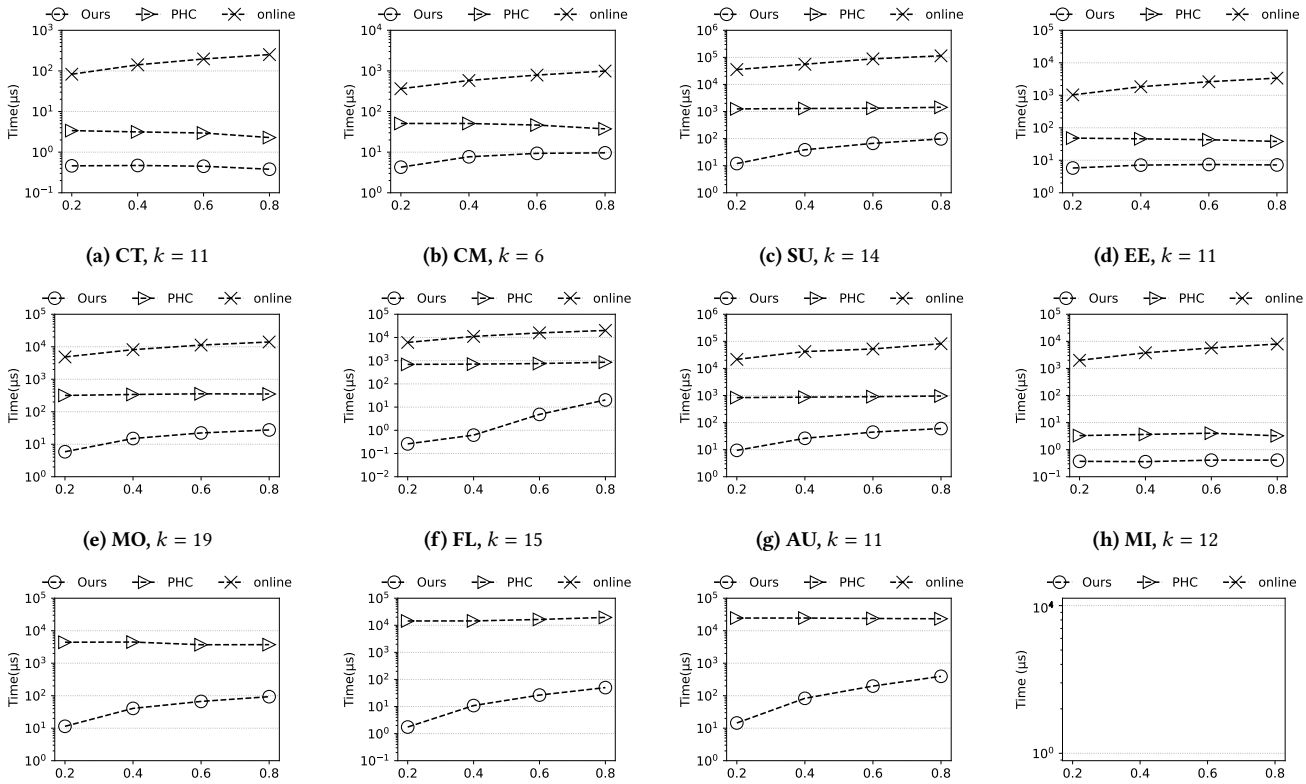
Index construction. We report the time cost required for constructing the index structures for the datasets used in our experiments. Note that the construction methods for Historical-Core queries build the structures for all values of k at once. The results are shown in Figure 12. Both construction algorithms **BTAS-Core** and **PHC** involve computing changes in vertex core numbers over time windows, which is the most time-consuming step, leading to similar execution time for both methods. Note that the result on the SO dataset is not included as we are unable to construct both index structures within 24 hours. Additionally, we measure and report the space consumption of index structure in Figure 13. Both index structures have the same space complexity, but **BTAS-Core** generally occupies slightly more space than **PHC**.

For the index maintenance, we remark that the state-of-the-art method **PHC** does not support the index maintenance, while our method **BTAS-Core** is capable of handling dynamic updates in theory. However, the current construction and maintenance of our index structure are based on the core intermediate data computed by **PHC**. Since **PHC** lacks support for dynamic updates, our index structure also inherits this limitation, though alternative methods can be explored for handling updates in the future.

5 RELATED WORK

Historical-CSM queries in temporal graphs. The problem of Historical-CSM queries in temporal graphs has attracted tremendous attention, which aims to design indices for efficient retrieval of CSMs within an arbitrary time interval. The k-core model has been a central topic in the study of Historical-CSM queries. Yu et al. [59] introduced the problem of historical k-core and proposed the PHC-Index to efficiently address the problem. Later, Yang et al. [56] developed the EF-index to support the query of historical k-core component, given a specific vertex and time interval. Additionally, Tian et al. [46] investigated the historical (α, β)-core model in temporal bipartite graphs. Besides core-like structures, Xie et al. [54] focused on the (strongly) connected components and studied the corresponding historical queries in temporal graphs. Recently, Chen et al. [10] considered the historical queries for maximal clique.

Other CSM problems in temporal graphs. There also exist other CSM problems with different problem goals in temporal graphs. Yang et al. [55] and Zhong et al. [61] developed efficient online algorithms to identify all distinct historical k-cores, as well as k-cores that meet specific constraining conditions, across various time intervals. Galimberti et al. [18] introduced the concept of span-core, where two vertices must interact throughout the entire interval, and developed a core decomposition algorithm for this model. Similarly, Lotito and Montresor [30] introduced the concept of span-truss. Other studies have explored various time-relevant metrics for k-cores in temporal graphs, such as interaction frequency [2, 52], persistence [25], burstiness [38], periodicity [39, 60], continuity [27], and reliability [45]. Besides, Guo and Sekerinski



(a) CT, $k = 11$ (b) CM, $k = 6$ (c) SU, $k = 14$ (d) EE, $k = 11$

(e) MO, $k = 19$ (f) FL, $k = 15$ (g) AU, $k = 11$ (h) MI, $k = 12$

(i) WT, $k = 34$ (j) YG, $k = 26$ (k) WG, $k = 61$ (l) SO, $k = 45$

Figure 11: Average time cost per query with $k = 30\% k_{\max}$ for Historical-Core

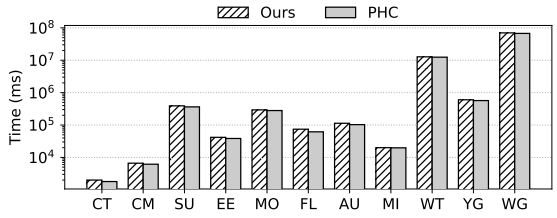


Figure 12: Index construction time for Historical-Core

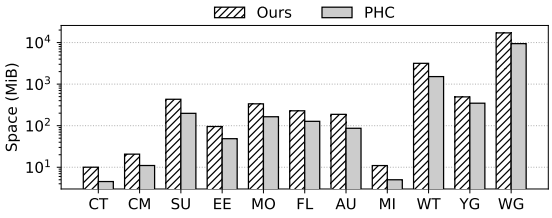


Figure 13: Index space consumption for Historical-Core

[19] studied how to maintain the core structure when a new edge is appended in the temporal setting, while Yang et al. [57] studied the quasi-clique in temporal graphs.

There are some other related studies include (1) the CSM-related problems on static graphs (e.g., k-plex [7, 8], k-core [3, 12], k-truss [14, 22, 47], γ-quasi-clique [58], k-defective clique [6, 16, 17], and densest subgraph [31]); and (2) queries (that are not related to CSMs) on temporal graphs (e.g., shortest path [51], historical connectivity between two vertices [44], reachability [42, 50], shortest path [51], communication motifs [20], and structural diversity [11]).

6 CONCLUSION

In this paper, we studied the problem of historical non-overlapping and monotone cohesive subgraph models query (Historical-CSM), which includes the well-known Historical-CC and Historical-Core queries. We then proposed a unified algorithmic framework capable of constructing an index structure **BTAS** specifically designed to support different Historical-CSM queries. Extensive experiments demonstrate the practical superiority of our method **BTAS** over the existing state-of-the-art methods for Historical-CC and Historical-Core queries. In the future, it is interesting to explore the adaptability of **BTAS** to other CSMs.

REFERENCES

[1] Technical Report and Source Code. https://anonymous.4open.science/r/Querying-Cohesive-Subgraphs-in-Temporal-Graphs-C38D.

[2] Wen Bai, Yadi Chen, and Di Wu. 2020. Efficient Temporal Core Maintenance of Massive Graphs. *Information Sciences* 513 (2020), 324–340.

[3] Vladimir Batagelj and Matjaž Zaveršnik. 2003. An $O(m)$ Algorithm for Cores Decomposition of Networks. *CoRR* cs.DS/0310049 (2003).

[4] Larissa M Batrancea, Ömer Akgüller, Mehmet Ali Balcı, and Anca Nichita. 2024. Financial Network Communities and Methodological Insights: A Case Study for Borsa Istanbul Sustainability Index. *Humanities and Social Sciences Communications* 11, 1 (2024), 1–27.

[5] Ulrik Brandes. 2005. *Network Analysis: Methodological Foundations*. Springer Science & Business Media.

[6] Lijun Chang. 2023. Efficient Maximum k-Defective Clique Computation with Improved Time Complexity. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 3 (2023), 1–26.

[7] Lijun Chang, Mouyi Xu, and Darren Strash. 2022. Efficient Maximum k-Plex Computation over Large Sparse Graphs. *Proceedings of the VLDB Endowment* 16, 2 (2022), 127–139.

[8] Lijun Chang and Kai Yao. 2024. Maximum k-Plex Computation: Theory and Practice. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 1 (2024), 1–26.

[9] Bernard Chazelle. 1990. Lower Bounds for Orthogonal Range Searching: Part II. The Arithmetic Model. *J. ACM* 37 (1990), 439–463.

[10] Kaiyu Chen, Dong Wen, Wentao Li, Zhengyi Yang, and Wenjie Zhang. 2024. On Compressing Historical Cliques in Temporal Graphs. In *Proceedings of the International Conference on Database Systems for Advanced Applications (DASFAA)*. 37–53.

[11] Kaiyu Chen, Dong Wen, Wenjie Zhang, Ying Zhang, Xiaoyang Wang, and Xuemin Lin. 2024. Querying Structural Diversity in Streaming Graphs. *Proceedings of the VLDB Endowment* 17, 5 (2024), 1034–1046.

[12] James Cheng, Yiping Ke, Shumo Chu, and M Tamer Özsu. 2011. Efficient Core Decomposition in Massive Networks. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 51–62.

[13] Martino Ciaperoni, Edoardo Galimberti, Francesco Bonchi, Ciro Cattuto, Francesco Gullo, and Alain Barrat. 2020. Relevance of Temporal Cores for Epidemic Spread in Temporal Networks. *Scientific reports* 10, 1 (2020), 12529.

[14] Jonathan Cohen. 2008. Trusses: Cohesive Subgraphs for Social Network Analysis. *National Security Agency Technical Report* 16, 3.1 (2008).

[15] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. 2022. *Introduction to Algorithms*. MIT press.

[16] Qiangqiang Dai, Ronghua Li, Donghang Cui, and Guoren Wang. 2024. Theoretically and Practically Efficient Maximum Defective Clique Search. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 4 (2024), 1–27.

[17] Qiangqiang Dai, Rong-Hua Li, Meihao Liao, and Guoren Wang. 2023. Maximal Defective Clique Enumeration. *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)* 1, 1 (2023), 1–26.

[18] Edoardo Galimberti, Alain Barrat, Francesco Bonchi, Ciro Cattuto, and Francesco Gullo. 2018. Mining (Maximal) Span-Cores from Temporal Networks. In *Proceedings of the ACM International Conference on Information and Knowledge Management (CIKM)*. 107–116.

[19] Bin Guo and Emil Sekerinski. 2024. Simplified Algorithms for Order-Based Core Maintenance. *The Journal of Supercomputing* 80, 13 (2024), 19592–19623.

[20] Saket Gurukar, Sayan Ranu, and Balaraman Ravindran. 2015. Commit: A Scalable Approach to Mining Communication Motifs from Dynamic Networks. In *Proceedings of the ACM SIGMOD International Conference on Management of Data (SIGMOD)*. 475–489.

[21] Chuhan Hu, Ming Zhong, Yuanyuan Zhu, Tieyun Qian, Ting Yu, Hongyang Chen, Mengchi Liu, and X Yu Jeffrey. 2024. Querying Cohesive Subgraph Regarding Span-Constrained Triangles on Temporal Graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 3338–3350.

[22] Xin Huang, Hong Cheng, Lu Qin, Wentao Tian, and Jeffrey Xu Yu. 2014. Querying k-Truss Community in Large and Dynamic Graphs. In *Proceedings of the ACM on Management of Data (SIGMOD)*. 1311–1322.

[23] Wissam Khaouid, Marina Barsky, Venkatesh Srinivasan, and Alex Thomo. 2015. K-Core Decomposition of Large Networks on a Single PC. *Proceedings of the VLDB Endowment* 9, 1 (2015), 13–23.

[24] Udayan Khurana and Amol Deshpande. 2013. Efficient snapshot retrieval over historical graph data. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 997–1008.

[25] Rong-Hua Li, Jiao Su, Lu Qin, Jeffrey Xu Yu, and Qiangqiang Dai. 2018. Persistent Community Search in Temporal Networks. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 797–808.

[26] Shunyang Li, Kai Wang, Xuemin Lin, Wenjie Zhang, Yizhang He, and Long Yuan. 2024. Querying Historical Cohesive Subgraphs over Temporal Bipartite Graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 2503–2516.

[27] Yuan Li, Jinsheng Liu, Huiqun Zhao, Jing Sun, Yuhai Zhao, and Guoren Wang. 2021. Efficient Continual Cohesive Subgraph Search in Large Temporal Graphs. *World Wide Web* 24 (2021), 1483–1509.

[28] Longlong Lin, Pingpeng Yuan, Rong-Hua Li, Chunxue Zhu, Hongchao Qin, Hai Jin, and Tao Jia. 2024. QTCS: Efficient Query-Centered Temporal Community Search. *Proceedings of the VLDB Endowment* 17, 6 (2024), 1187–1199.

[29] Boge Liu, Long Yuan, Xuemin Lin, Lu Qin, Wenjie Zhang, and Jingren Zhou. 2020. Efficient (α, β)-Core Computation in Bipartite Graphs. *The VLDB Journal* 29, 5 (2020), 1075–1099.

[30] Quintino Francesco Lotito and Alberto Montresor. 2020. Efficient Algorithms to Mine Maximal Span-Trusses From Temporal Graphs. *arXiv preprint arXiv:2009.01928* (2020).

[31] Chenhao Ma, Yixiang Fang, Reynold Cheng, Laks VS Lakshmanan, Wenjie Zhang, and Xuemin Lin. 2021. On Directed Densest Subgraph Discovery. *ACM Transactions on Database Systems (TODS)* 46, 4 (2021), 1–45.

[32] Qiuyang Mang, Jingbang Chen, Hangrui Zhou, Yu Gao, Yingli Zhou, Richard Peng, Yixiang Fang, and Chenhao Ma. 2024. Efficient Historical Butterfly Counting in Large Temporal Bipartite Networks via Graph Structure-aware Index. *arXiv preprint arXiv:2406.00344* (2024).

[33] Naoki Masuda and Renaud Lambiotte. 2016. *A Guide to Temporal Networks*. World Scientific.

[34] Edward M. McCreight. 1985. Priority Search Trees. *SIAM J. Comput.* 14, 2 (1985), 257–276.

[35] Mark Newman. 2010. *Networks: An Introduction*. OUP Oxford.

[36] Vladimír Pažitka, Michael Urban, and Dariusz Wójcik. 2021. Connectivity and Growth: Financial Centres in Investment Banking Networks. *Environment and Planning A: Economy and Space* 53, 7 (2021), 1789–1809.

[37] Hongchao Qin, Rong-Hua Li, Ye Yuan, Yongheng Dai, and Guoren Wang. 2023. Densest Periodic Subgraph Mining on Large Temporal Graphs. *IEEE Transactions on Knowledge and Data Engineering* 35, 11 (2023), 11259–11273.

[38] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Lu Qin, and Zhiwei Zhang. 2022. Mining Bursting Core in Large Temporal Graphs. *Proceedings of the VLDB Endowment* 15, 13 (2022), 3911–3923.

[39] Hongchao Qin, Rong-Hua Li, Ye Yuan, Guoren Wang, Weihua Yang, and Lu Qin. 2020. Periodic Communities Mining in Temporal Networks: Concepts and Algorithms. *IEEE Transactions on Knowledge and Data Engineering* 34, 8 (2020), 3927–3945.

[40] Chenghui Ren, Eric Lo, Ben Kao, Xinjie Zhu, and Reynold Cheng. 2011. On Querying Historical Evolving Graph Sequences. *Proceedings of the VLDB Endowment* 4, 11 (2011), 726–737.

[41] Konstantinos Semertzidis and Evaggelia Pitoura. 2016. Durable Graph Pattern Queries on Historical Graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 541–552.

[42] Konstantinos Semertzidis, Evaggelia Pitoura, and Kostas Lillis. 2015. TimeReach: Historical Reachability Queries on Evolving Graphs. In *Proceedings of the International Conference on Extending Database Technology (EDBT)*. 121–132.

[43] Daniel D. Sleator and Robert E. Tarjan. 1981. A Data Structure for Dynamic Trees. In *Proceedings of the Annual ACM Symposium on Theory of Computing (STOC)*. 114–122.

[44] Jingyi Song, Dong Wen, Lantian Xu, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2024. On Querying Historical Connectivity in Temporal Graphs. *Proceedings of the ACM on Management of Data (SIGMOD)* 2, 3 (2024), 1–25.

[45] Yifu Tang, Jianxin Li, Nur Al Hasan Haldar, Ziyu Guan, Jiajie Xu, and Chengfei Liu. 2022. Reliable Community Search in Dynamic Networks. *Proceedings of the VLDB Endowment* 15, 11 (2022), 2826–2838.

[46] Anxin Tian, Alexander Zhou, Yue Wang, Xun Jian, and Lei Chen. 2024. Efficient Index for Temporal Core Queries over Bipartite Graphs. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2813–2825.

[47] Jia Wang and James Cheng. 2012. Truss Decomposition in Massive Networks. *Proceedings of the VLDB Endowment* 5, 9 (2012), 812–823.

[48] Stanley Wasserman. 1994. Social Network Analysis: Methods and Applications. *The Press Syndicate of the University of Cambridge* (1994).

[49] Dong Wen, Yilun Huang, Ying Zhang, Lu Qin, Wenjie Zhang, and Xuemin Lin. 2020. Efficiently Answering Span-Reachability Queries in Large Temporal Graphs. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 1153–1164.

[50] Dong Wen, Bohua Yang, Ying Zhang, Lu Qin, Dawei Cheng, and Wenjie Zhang. 2022. Span-Reachability Querying in Large Temporal Graphs. *The VLDB Journal* 31, 4 (2022), 629–647.

[51] Huanhuan Wu, James Cheng, Silu Huang, Yiping Ke, Yi Lu, and Yanyan Xu. 2014. Path Problems in Temporal Graphs. *Proceedings of the VLDB Endowment* 7, 9 (2014), 721–732.

[52] Huanhuan Wu, James Cheng, Yi Lu, Yiping Ke, Yuzhen Huang, Da Yan, and Hejun Wu. 2015. Core Decomposition in Large Temporal Graphs. In *Proceedings of the IEEE International Conference on Big Data (BigData)*. 649–658.

[53] Ziyuan Xia, Anchen Sun, Jingyi Xu, Yuanzhe Peng, Rui Ma, and Minghui Cheng. 2022. Contemporary Recommendation Systems on Big Data and Their Applications: A Survey. *arXiv e-prints* (2022), arXiv–2206.

[54] Haoxuan Xie, Yixiang Fang, Yuyang Xia, Wensheng Luo, and Chenhao Ma. 2023. On Querying Connected Components in Large Temporal Graphs. *Proceedings of the ACM on Management of Data (SIGMOD)* 1, 2 (2023), 1–27.

[55] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tieyun Qian, Mengchi Liu, and Jeffery Xu Yu. 2023. Scalable Time-Range k-Core Query on Temporal Graphs. *Proceedings of the VLDB Endowment* 16, 5 (2023), 1168–1180.

[56] Junyong Yang, Ming Zhong, Yuanyuan Zhu, Tieyun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. Evolution Forest Index: Towards Optimal Temporal k-Core Component Search via Time-Topology Isomorphic Computation. *Proceedings of the VLDB Endowment* 17, 11 (2024), 2840–2853.

[57] Yi Yang, Da Yan, Huanhuan Wu, James Cheng, Shuigeng Zhou, and John CS Lui. 2016. Diversified Temporal Subgraph Pattern Mining. In *Proceedings of the ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (SIGKDD)*. 1965–1974.

[58] Kaiqiang Yu and Cheng Long. 2023. Fast Maximal Quasi-Clique Enumeration: A Pruning and Branching Co-Design Approach. *Proc. ACM SIGMOD Int. Conf. Manage. Data (SIGMOD)* 1, 3 (2023), 1–26.

[59] Michael Yu, Dong Wen, Lu Qin, Ying Zhang, Wenjie Zhang, and Xuemin Lin. 2021. On Querying Historical k-Cores. *Proceedings of the VLDB Endowment* 14, 11 (2021), 2033–2045.

[60] Yue Zeng, Hongchao Qin, Rong-Hua Li, Kai Wang, Guoren Wang, and Xuemin Lin. 2024. Mining Quasi-Periodic Communities in Temporal Network. In *Proceedings of the IEEE International Conference on Data Engineering (ICDE)*. 2476–2488.

[61] Ming Zhong, Junyong Yang, Yuanyuan Zhu, Tieyun Qian, Mengchi Liu, and Jeffrey Xu Yu. 2024. A Unified and Scalable Algorithm Framework of User-Defined Temporal $(k, \mathcal{X})$-Core Query. *IEEE Transactions on Knowledge and Data Engineering* 36, 7 (2024), 2831–2845.

A OMITTED PROOFS

A.1 Proof of Lemma 3.5

Before showing our proof of Lemma 3.5, we provide the following two lemmas.

Lemma A.1. *Let $G = (V, E)$ and $H = (V, E')$ be two graphs such that for any subset $S \subseteq V$, if $G[S] \in \mathcal{P}_{CC}(G)$, then $H[S] \in \mathcal{P}_{CC}$. Let $\overline{E}$ be an arbitrary edge set. Given $G^+ = (V, E \cup \overline{E})$ and $H^+ = (V, E' \cup \overline{E})$, we have that for any subset $S \subseteq V$, if $G^+[S] \in \mathcal{P}_{CC}(G^+)$, then $H^+[S] \in \mathcal{P}_{CC}(H^+)$.*

Proof. Consider two arbitrary vertexes $u, v \in V$ that are connected in G^+. By definition, there must exist a path between u and v in G^+ consisting of edges in $E \cup \overline{E}$. Suppose the path from u to v can be represented as (u, x_1), (x_1, x_2), (x_2, x_3), ..., (x_q, v). For any part of the path (x_i, x_{i+1}), (x_{i+1}, x_{i+2}), ..., (x_{j-1}, x_j) contained in E, we know x_i and x_j are connected in G. Given the condition that if $G[S] \in \mathcal{P}_{CC}(G)$, then $H[S] \in \mathcal{P}_{CC}(H)$. It follows that there also exists a path from x_i to x_j in E'. Thus, for each segment of the path in E, we can find a corresponding path in E'. By replacing each segment in E with the corresponding segment in E', we can construct a path between u and v using only edges from $E' \cup \overline{E}$, which shows that u and v are connected in H^+. Conversely, if two vertices are connected in H^+, the same argument applies in reverse, showing that they are also connected in G^+. Hence, for any subset $S \subseteq V$, if $G^+[S] \in \mathcal{P}_{CC}(G^+)$, then $H^+[S] \in \mathcal{P}_{CC}(H^+)$. We complete the proof of Lemma A.1. □

Lemma A.2. *Let $\mathcal{G} = (V, \mathcal{E})$ and $\mathcal{H} = (V, \mathcal{E}')$ be two temporal graphs such that for any subset $S \subseteq V$ and any time interval $[t_s, t_e]$, if $G_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(G_{[t_s,t_e]})$, then $H_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(H_{[t_s,t_e]})$. Let $\overline{\mathcal{E}}$ be an arbitrary spanning edge set, and we have $\mathcal{G}^+ = (V, \mathcal{E} \cup \overline{\mathcal{E}})$ and $\mathcal{H}^+ = (V, \mathcal{E}' \cup \overline{\mathcal{E}})$. It follows that for any subset $S \subseteq V$, if $G^+_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(G^+_{[t_s,t_e]})$, then $H^+_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(H^+_{[t_s,t_e]})$.*

Proof. For any time interval $[t_s, t_e]$, $G^+_{[t_s,t_e]} = (V, E_{[t_s,t_e]} \cup \overline{E}_{[t_s,t_e]})$ and $H^+_{[t_s,t_e]} = (V, E_{[t_s,t_e]} \cup \overline{E}_{[t_s,t_e]})$. According to Lemma A.1, we have: for any subset $S \subseteq V$, if $G^+_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(G^+_{[t_s,t_e]})$, then $H^+_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(H^+_{[t_s,t_e]})$, completing the proof. □

We are ready to prove and restate Lemma 3.5 below.

Lemma A.3. *Let $\overline{\mathcal{G}}$ be an affiliated graph and $(V, \overline{\Psi})$ be a refined one of $\overline{\mathcal{G}}$. For any time window $[t_s, t_e]$, $(V, \overline{\Psi}_{\langle t_s,t_e \rangle})$ is a refined affiliated graph of $\overline{\mathcal{G}}_{\langle t_s,t_e \rangle}$.*

Proof. Suppose to the contrary that $(V, \overline{\Psi}_{\langle t_s,t_e \rangle})$ is not a refined affiliated graph of $\overline{\mathcal{G}}_{\langle t_s,t_e \rangle}$. There must exist a refined affiliated graph $(V, \overline{\Psi}')$ of $\overline{\mathcal{G}}_{\langle t_s,t_e \rangle}$, which has fewer edges, i.e., $|\overline{\Psi}'| < |\overline{\Psi}_{\langle t_s,t_e \rangle}|$. Consider the edge set $\overline{\Psi}'' = \overline{\Psi}' \cup (\overline{\Psi} \setminus \overline{\Psi}_{\langle t_s,t_e \rangle}$. For any spanning edge (u, v, t_1, t_2) in $\overline{\Psi} \setminus \overline{\Psi}_{\langle t_s,t_e \rangle}$, the condition $[t_1, t_2] \not\subseteq [t_s, t_e]$ always holds, hence $\overline{\Psi}' \cap (\overline{\Psi} \setminus \overline{\Psi}_{\langle t_s,t_e \rangle}) = \emptyset$. It follows that $|\overline{\Psi}''| = |\overline{\Psi}'| + |\overline{\Psi}| - |\overline{\Psi}_{\langle t_s,t_e \rangle}| < |\overline{\Psi}|$. Note that $\overline{\Psi}$ can be expressed as $\overline{\Psi} = \overline{\Psi}_{\langle t_s,t_e \rangle} \cup (\overline{\Psi} \setminus \overline{\Psi}_{\langle t_s,t_e \rangle})$. Let $\overline{\mathcal{G}}' = (V, \overline{\Psi})$ and $\overline{\mathcal{G}''} = (V, \overline{\Psi}'')$. According to Lemma A.2, we have: for any subgraph $S \subseteq V$, if $G'_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(G'_{[t_s,t_e]})$, then $G''_{[t_s,t_e]}[S] \in \mathcal{P}_{CC}(G''_{[t_s,t_e]})$.

Algorithm 6: `PST-Construction`

Input: The 2-D point set P
Output: A priority search tree T for P

```
T ← ∅;
p ← the point in P with the minimum y-coordinates;
P ← P \ {p};
x_med ← the median of x-coordinates of points in P;
P_left ← {(x, y) ∈ P : x < x_med};
P_right ← {(x, y) ∈ P : x > x_med};
left sub-tree of T ← PST-Construction(P_left);
right sub-tree of T ← PST-Construction(P_right);
```

Thus, $\mathcal{G}''$ is also a refined affiliated graph of $\overline{\mathcal{G}}$, but with fewer edges than $\mathcal{G}'$, which contradicts the assumption that $\mathcal{G}' = (V, \overline{\Psi})$ is a refined affiliated graph of $\overline{\mathcal{G}}$, completing the proof. □

A.2 Proof of Lemma 3.10

Proof. The **BTAS** structure is built recursively. For each recursion, the time complexity is dominated by the index construction time of the priority search tree T_{xy} and T_{xz} in Lines 9-10 of Algorithm 5. According to [34], the priority search trees T_{xy} and T_{xz} with $|P_{mid}|$ data points take $O(|P_{mid}| \log |P_{mid}|)$ time. Hence, the time in each recursion is $O(n + |P_{mid}| \log |P_{mid}|)$. Then, it is easy to see the total complexity of Algorithm 5 is $O(n \log n)$.

For the space complexity, we can observe that (1) each data point is only stored in one tree node in a **BTAS** tree, and (2) each data point is stored at most twice in the two priority search trees. Thus, the total space complexity is $O(n)$. □

B DESCRIPTION OF PRIORITY SEARCH TREE

For completeness, we provide the related operations of priority search tree (PST) [34].

Given a 2-D point set P, the priority search tree can search points $(x, y) \in P$ that satisfy $x_{q1} \leq x \leq x_{q2} \wedge y_{q2} \leq y$. The standard procedures for the PST construction and query are shown in Algorithm 6 and Algorithm 7, respectively. We note that PST can also support three-sided queries such as $x_{q1} \leq x \leq x_{q2} \wedge y \leq y_{q2}$ in a similar manner. We also note that with a set of n 2-D data points, the corresponding PST can be constructed in $O(n \log n)$ time and answers each query in $O(k + \log n)$ time, where k is the number of returned data points [34].

C ADDITIONAL EXPERIMENTAL STUDIES

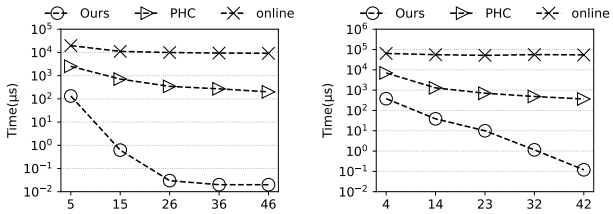
We first conduct the experiments by varying the values of k on the FL and SU datasets (shown in Figure 14). We can conclude that the query time of all indices decreases monotonically with increasing k, since a larger k implies fewer vertices in the resulting k-core.

We next provide additional experimental results for omitted values of k when k = 10%, 30%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core queries. Note that we focus on the experiments for query processing, since the index construction algorithms for Historical-Core queries build the index with all values of k. The results are illustrated in Figure 15 to Figure 25. We can conclude that our algorithm consistently outperforms the **PHC** and **online**

Algorithm 7: `PST-Query`

Input: The priority search tree T and the query parameters x_{q1}, x_{q2}, y_q
Output: The result set R of data points

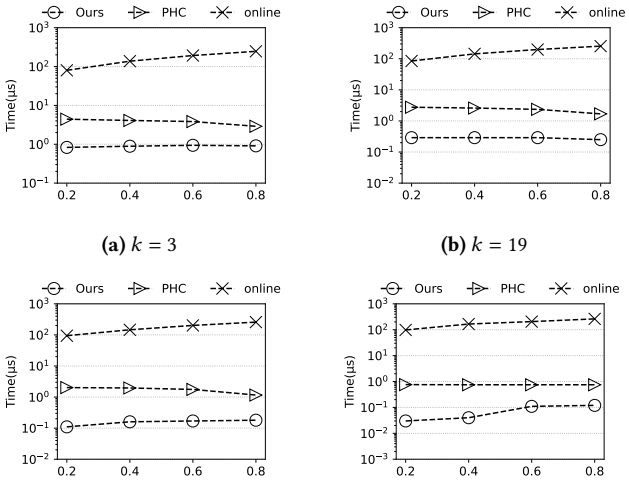
$R \leftarrow \emptyset$;
if $T = \emptyset$ **then**
  return R;
$(x, y) \leftarrow$ the point with minimum y stored in the root node;
$x_{med} \leftarrow$ the median of x stored in the root node;
if $y > y_q$ **then**
  return R;
if $x_{q1} \leq x \leq x_{q2}$ **then**
  $R \leftarrow R \cup (x, y)$;
if $x_{q1} \leq x_{med}$ **then**
  $R \leftarrow R \cup$ `PST-Query`$(T_{left}, x_{q1}, x_{q2}, y_q)$;
if $x_{med} \leq x_{q_2}$ **then**
  $R \leftarrow R \cup$ `PST-Query`$(T_{right}, x_{q1}, x_{q2}, y_q)$;
return R;



(a) FL **(b) SU**

Figure 14: Average time cost per query with the query interval of $30\%t_{\max}$ for Historical-Core

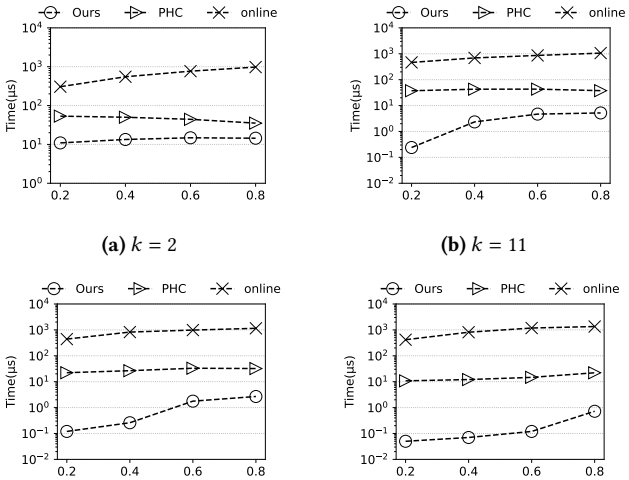
methods across various parameter settings. Additionally, as k increases, the query performance of ours and **PHC** indexing structures improves, since the resulting k-core involves fewer vertices. For example, on the WG dataset (in Figure 25), the **BTAS** can solve queries within 1 microsecond (10^{-6} seconds) when $k = 185$, which is significantly faster (around 874,000× faster) than the **PHC** index.



(a) $k = 3$ **(b)** $k = 19$

(c) $k = 27$ **(d)** $k = 35$

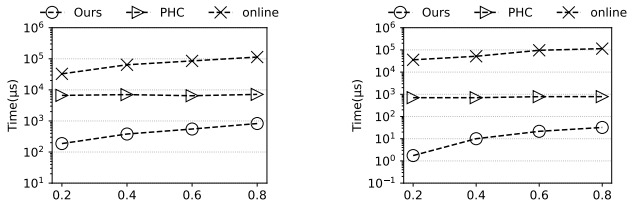
Figure 15: CT with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core



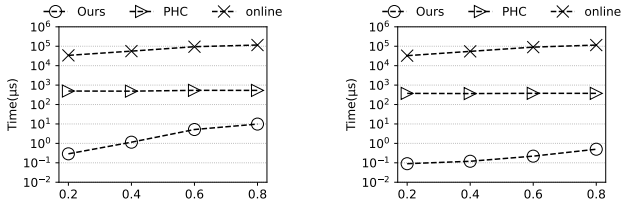
(a) $k = 2$ **(b)** $k = 11$

(c) $k = 15$ **(d)** $k = 19$

Figure 16: CM with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core

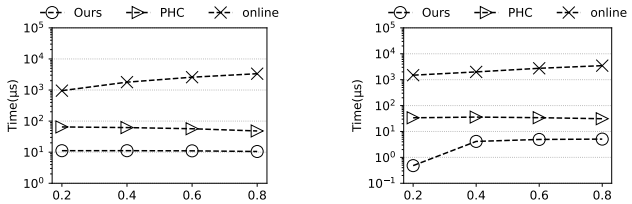


(a) $k = 4$ **(b)** $k = 23$

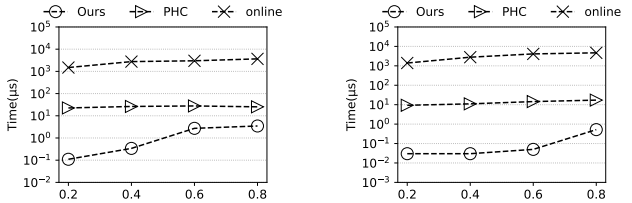


(c) $k = 32$ **(d)** $k = 42$

Figure 17: SU with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core

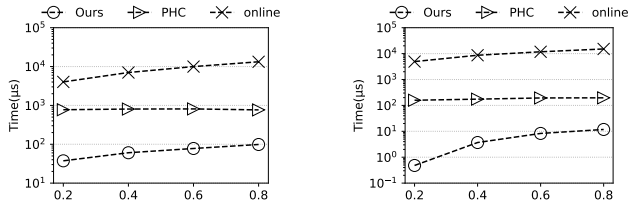


(a) $k = 3$ **(b)** $k = 19$

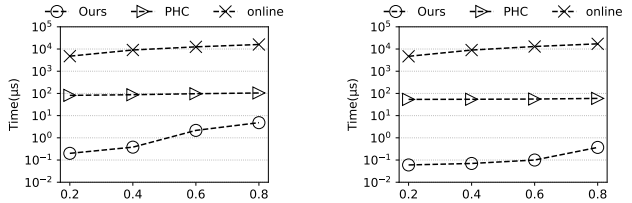


(c) $k = 26$ **(d)** $k = 34$

Figure 18: EE with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core

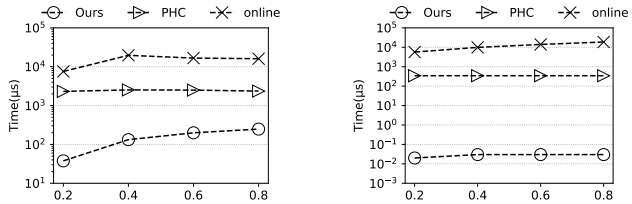


(a) $k = 6$ **(b)** $k = 33$

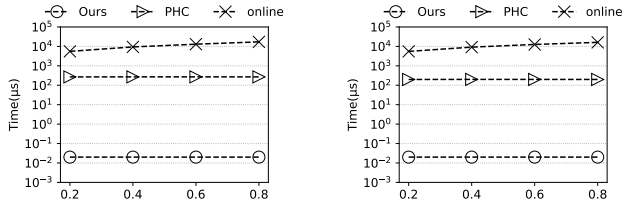


(c) $k = 46$ **(d)** $k = 59$

Figure 19: MO with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core

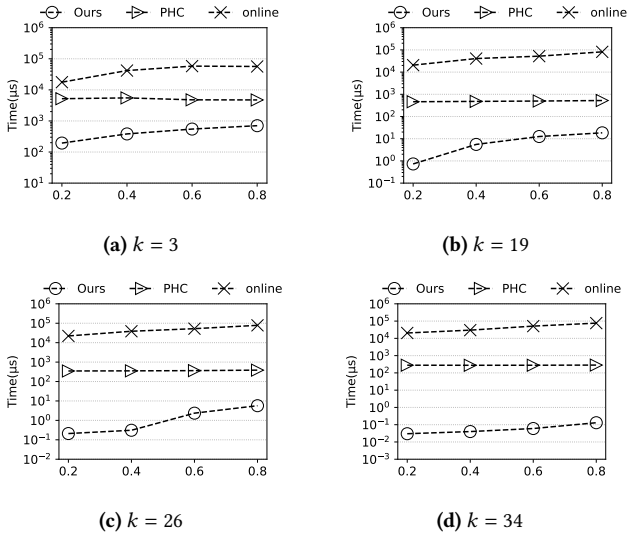


(a) $k = 5$ **(b)** $k = 26$



(c) $k = 36$ **(d)** $k = 46$

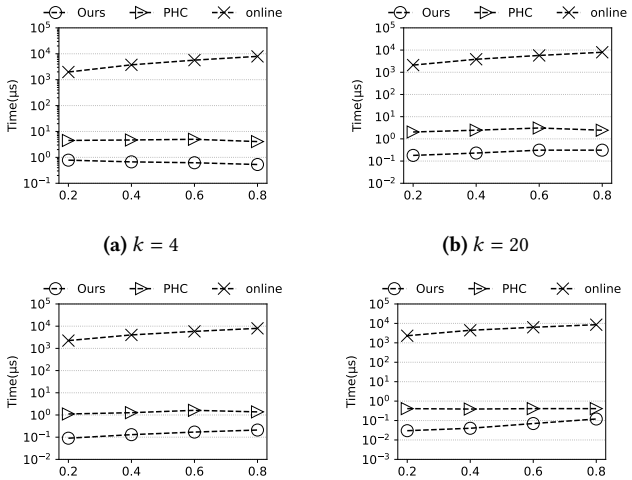
Figure 20: FL with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core



(a) $k = 3$ **(b)** $k = 19$

(c) $k = 26$ **(d)** $k = 34$

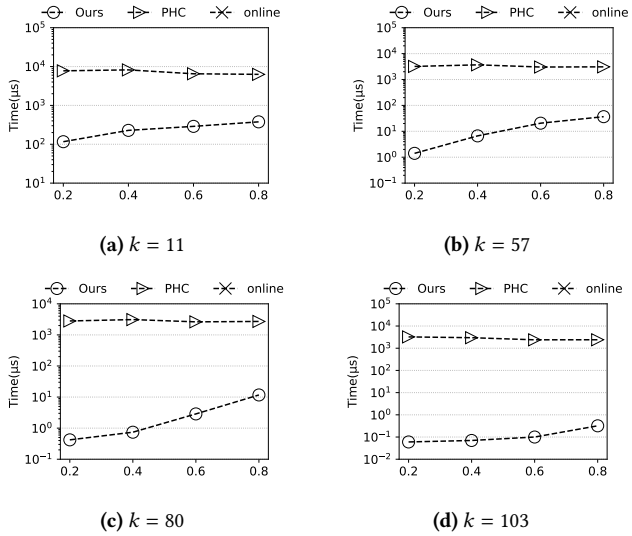
Figure 21: AU with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core



(a) $k = 4$ **(b)** $k = 20$

(c) $k = 28$ **(d)** $k = 36$

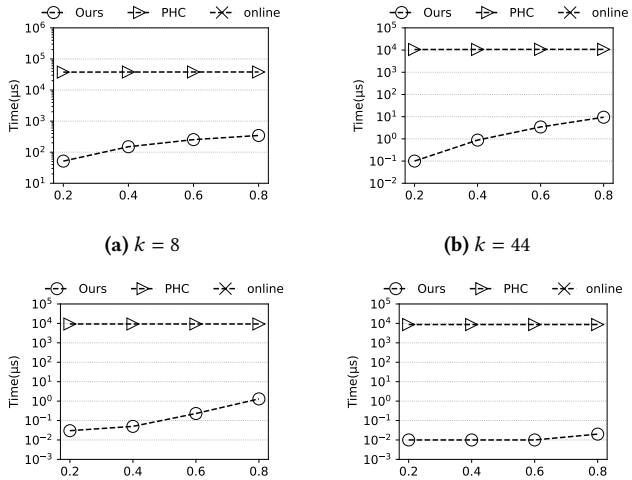
Figure 22: MI with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core



(a) $k = 11$ **(b)** $k = 57$

(c) $k = 80$ **(d)** $k = 103$

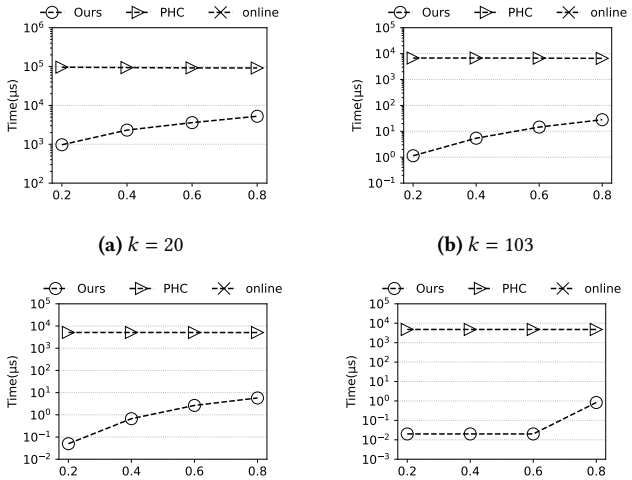
Figure 23: WT with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core



(a) $k = 8$ **(b)** $k = 44$

(c) $k = 61$ **(d)** $k = 79$

Figure 24: YG with k = 10%, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core



(a) $k = 20$ **(b)** $k = 103$

(c) $k = 144$ **(d)** $k = 185$

Figure 25: WG with $k = 10\%$, 50%, 70%, and 90% of $k_{\max}$ for Historical-Core